

Printer Dover

Spruce version 11.2 -- spooler version 11.2

File: ProcH-Rev-Ce.sil etc.

Creation date: 7-Jul-79 22:05:27

For: Spruce

34 total sheets = 33 pages, 1 copy.

Problems encountered:

Font HELVETICA24B substituted for font HELVE24.

Character code 117b not found in font Gates32.

Character code 157b not found in font Gates32.

Character code 157b not found in font Gates32.

Character code 117b not found in font Gates32.

Character code 117b not found in font Gates32.

Character code 157b not found in font Gates32.

Character code 157b not found in font Gates32.

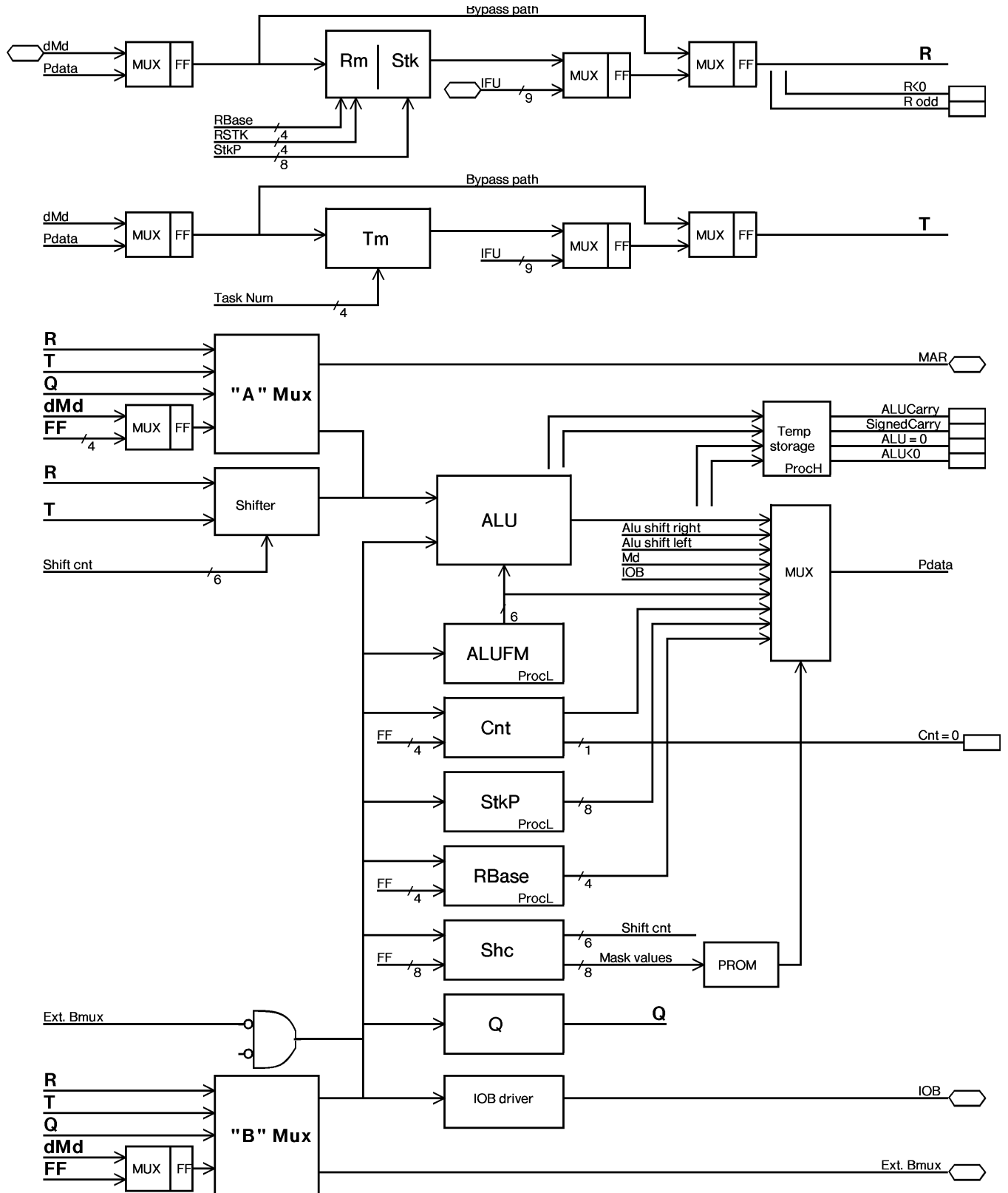
DORADO SCHEMATICS

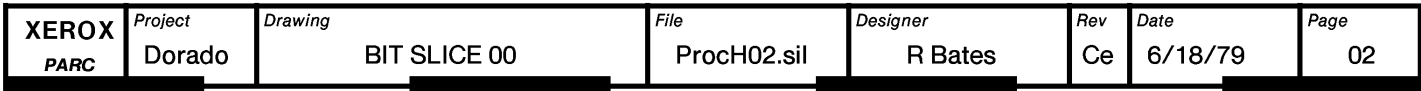
Hi Byte

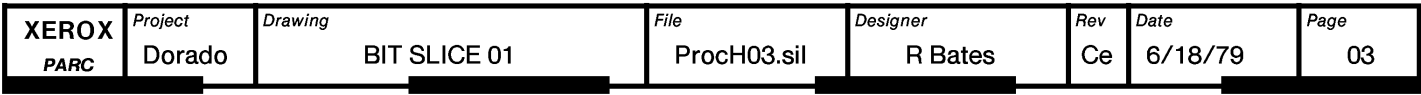
PROCESSOR

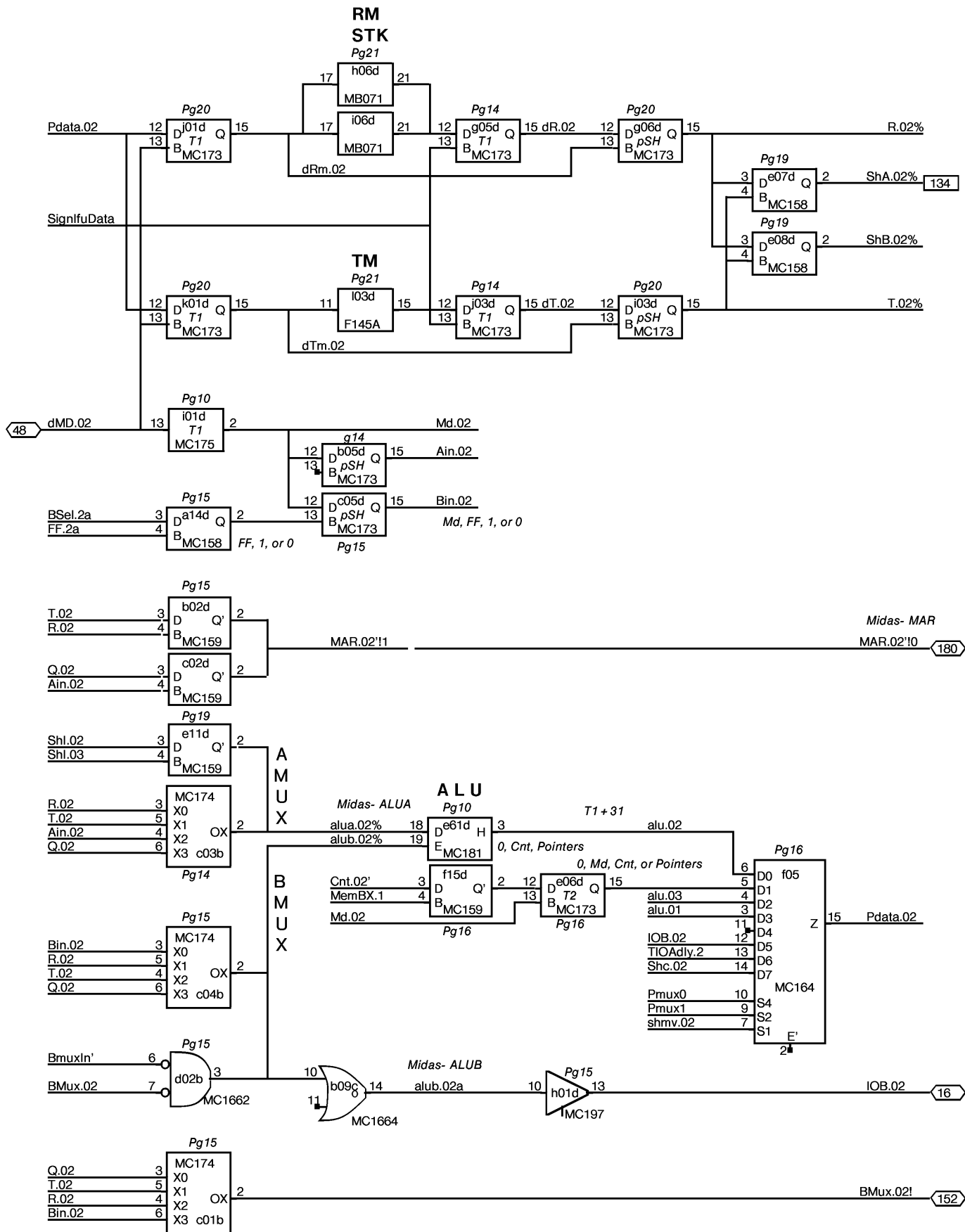
Table of contents

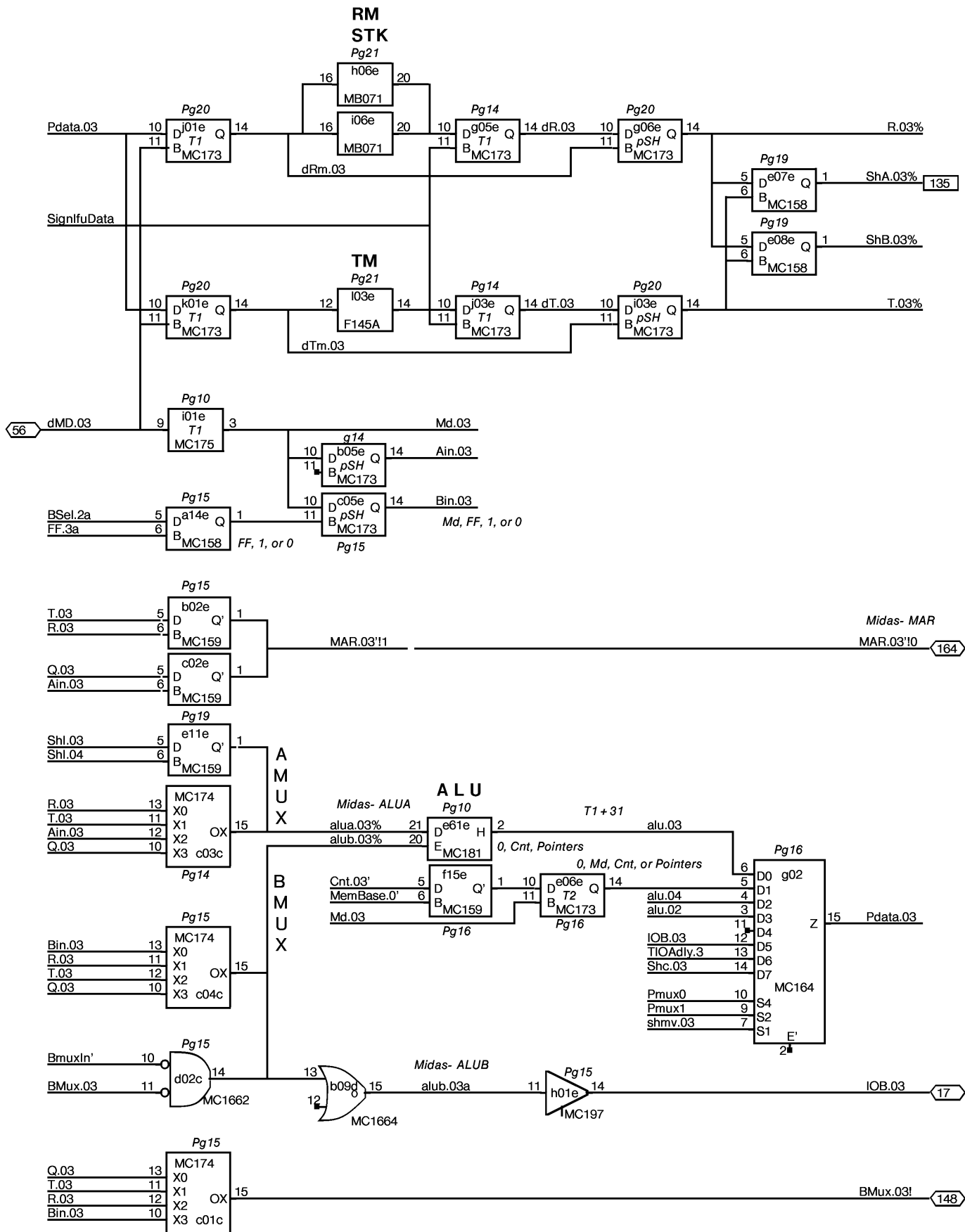
<u>TITLE</u>	<u>Page</u>
Bit slice drawings	02
ALU carry and RM fast-branch	10
Branch condition Ram	11
Task number logic	12
Function Field decoding	13
"A" multiplexor & Mar multiplexor control	14
"B" multiplexor control	15
"Processor" multiplexor control	16
Q and Count Registers	17
Shift Control register	18
Shifter multiplexors	19
Rm & T write control	20
Rm, Stk, and T address lines	21
Rm control	22
TIOA Ram	23
MemBase Register	24
Task Simulator and Temperature Sense	26
Parity logic	27
Midas control and multiplexors	28
Clock distribution	30
Layout	31
Loading Information	32

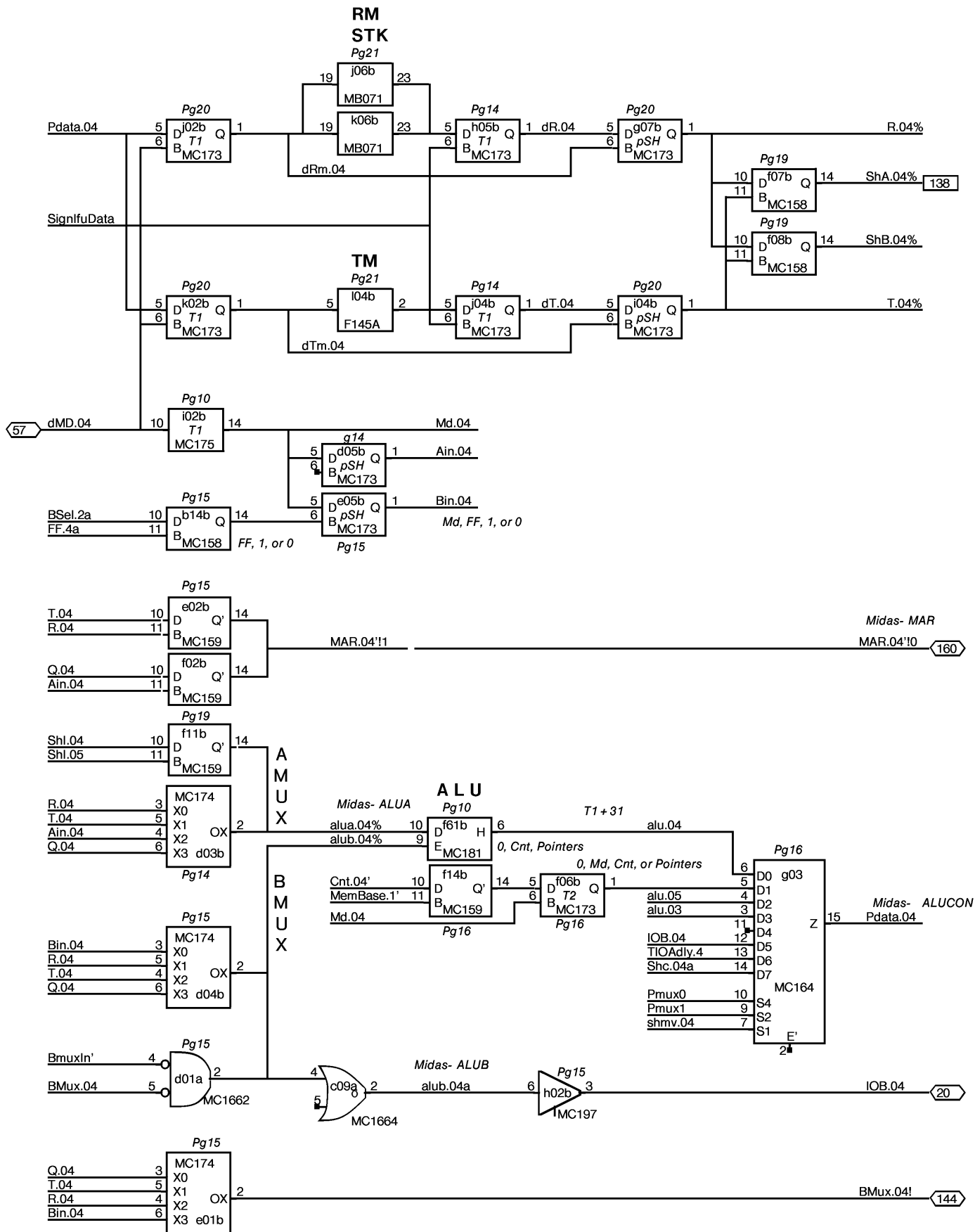


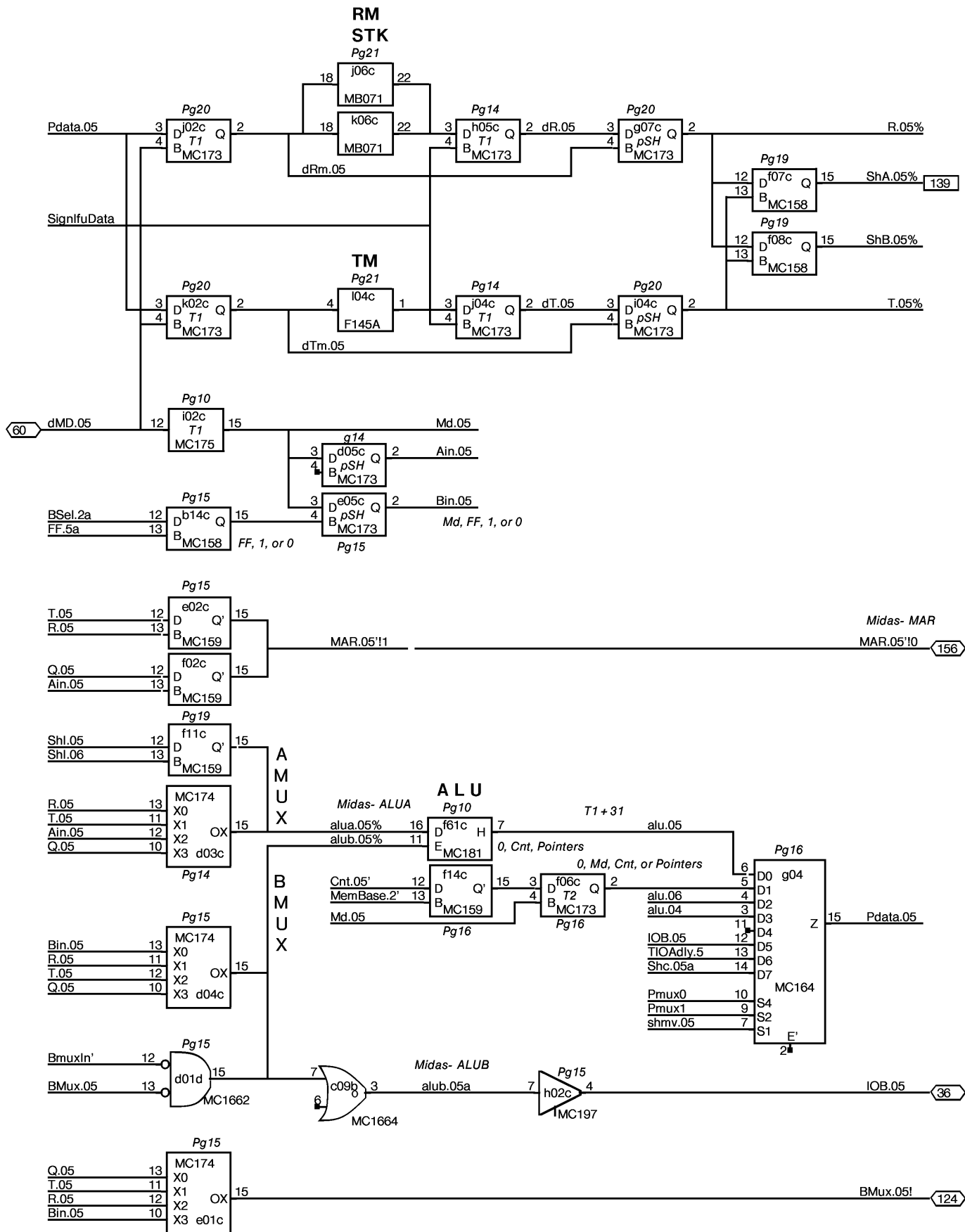


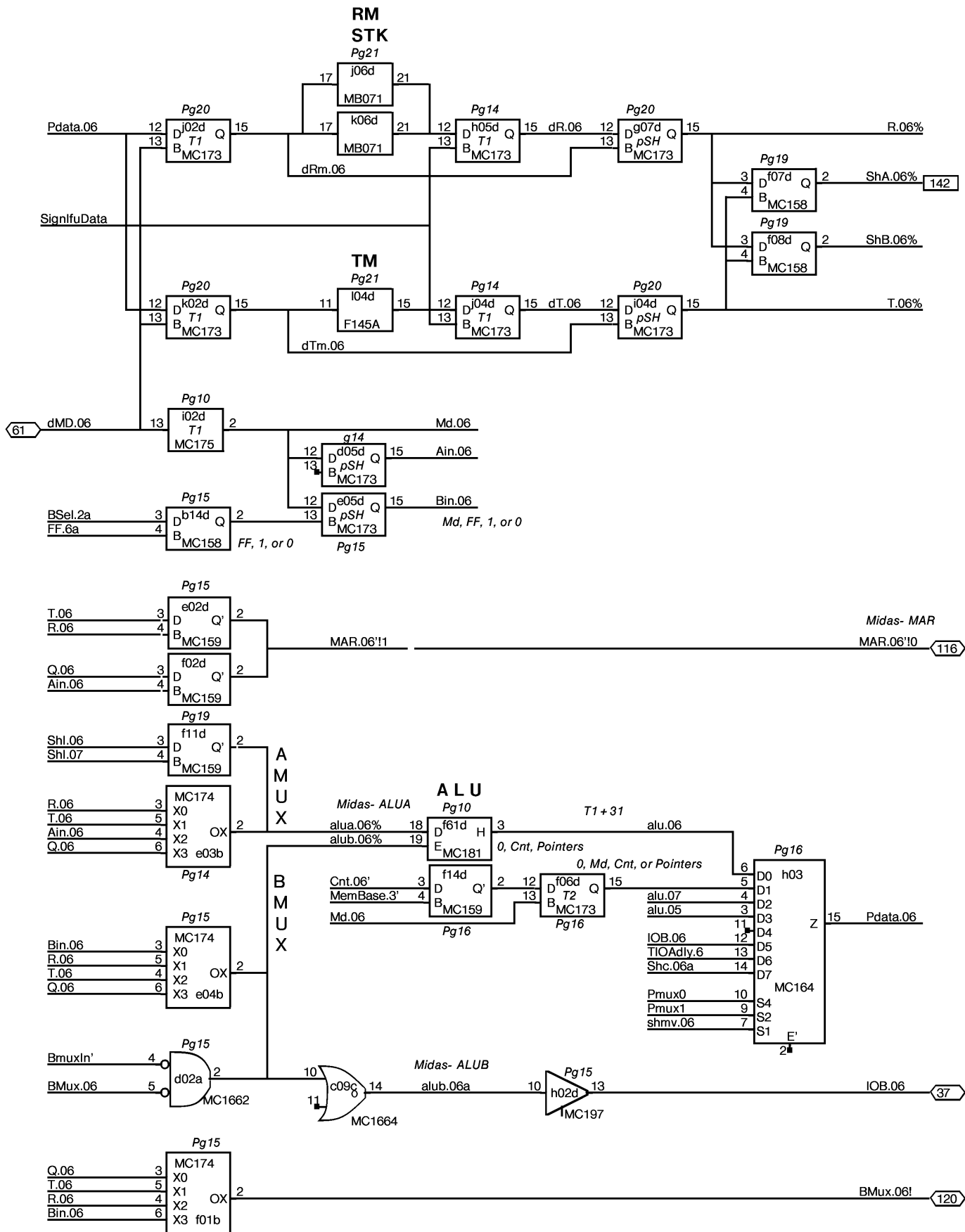


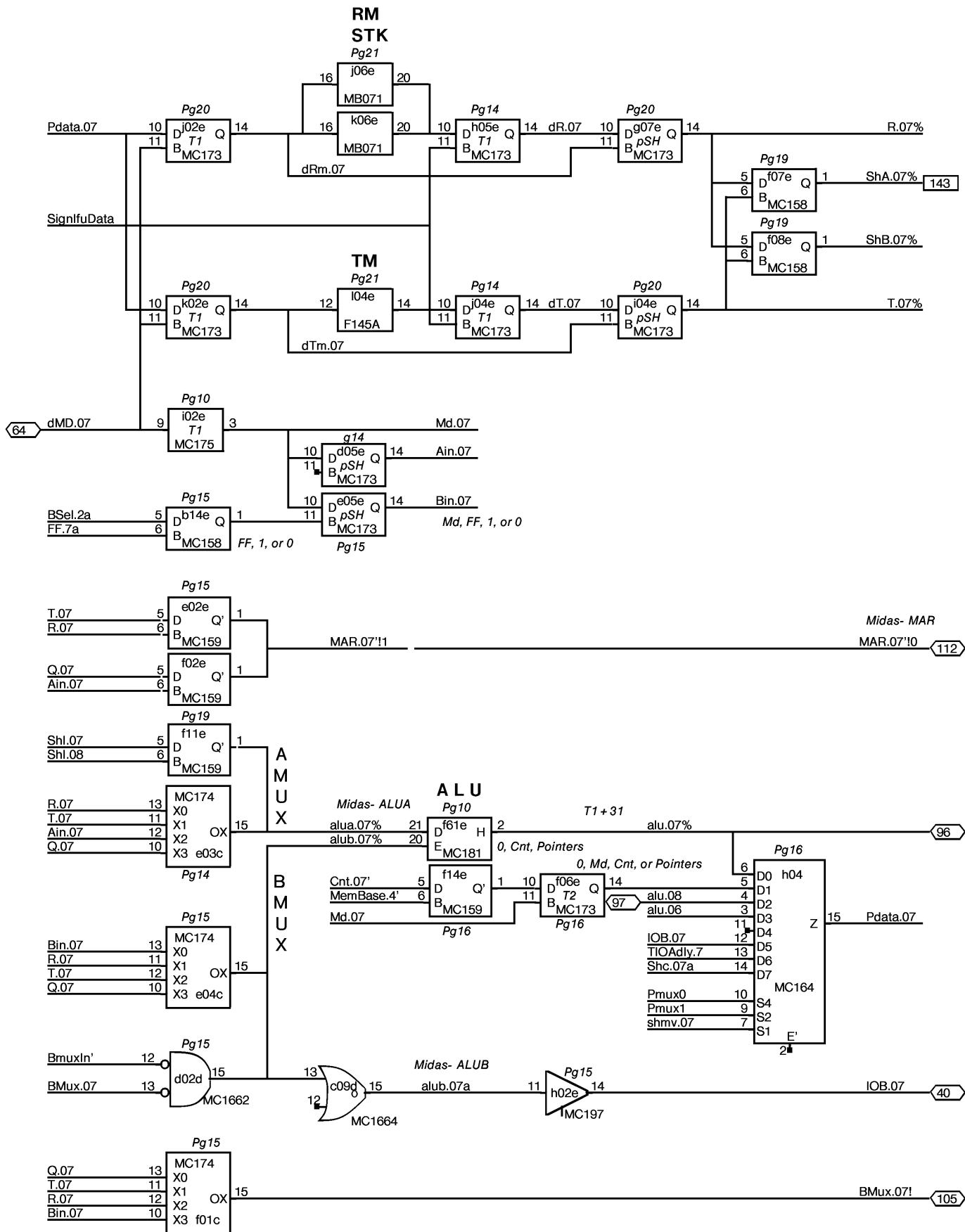












ALU delays

Logical function to output = 11.9

Arithmetic operation to data = 20.0

Arithmetic operation to carry = 17.9

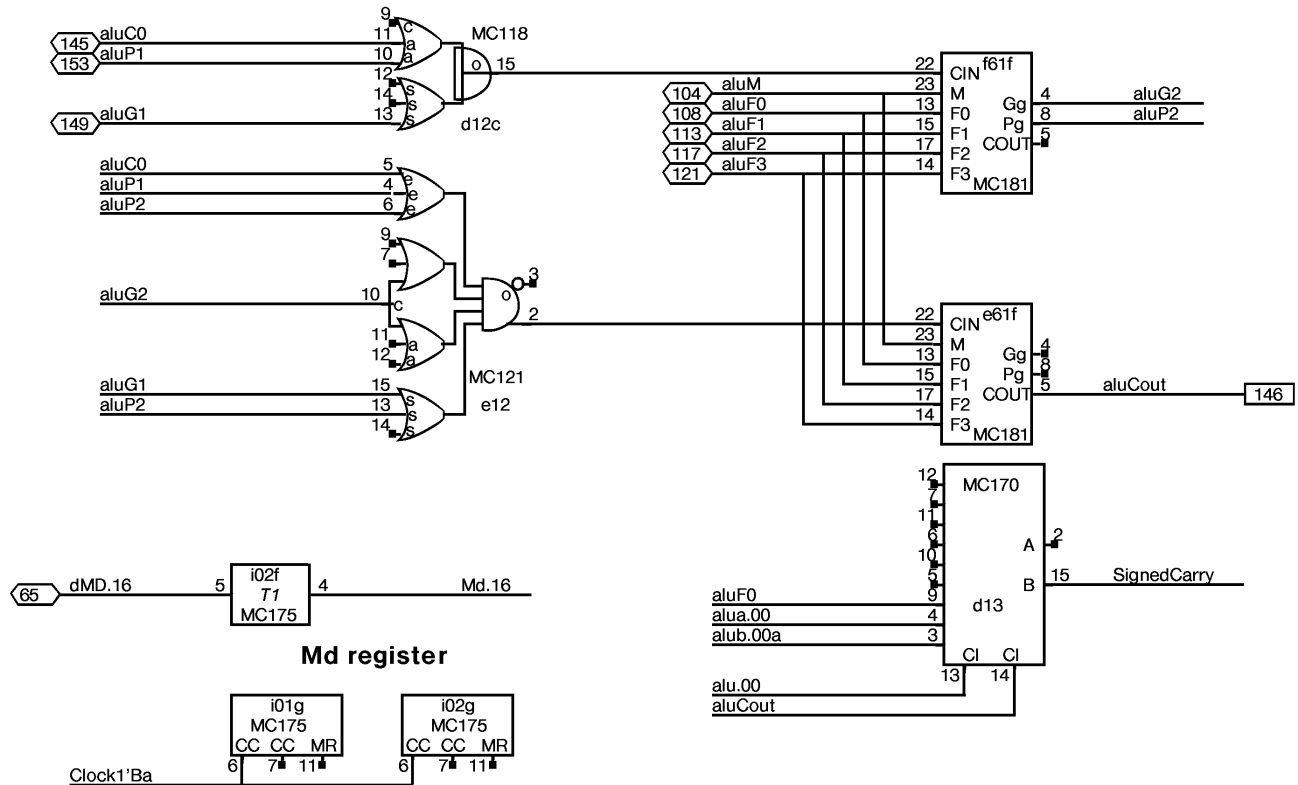
ALU output

(assuming 10.2 ns to output of BMux)

Logical Function = 22.0

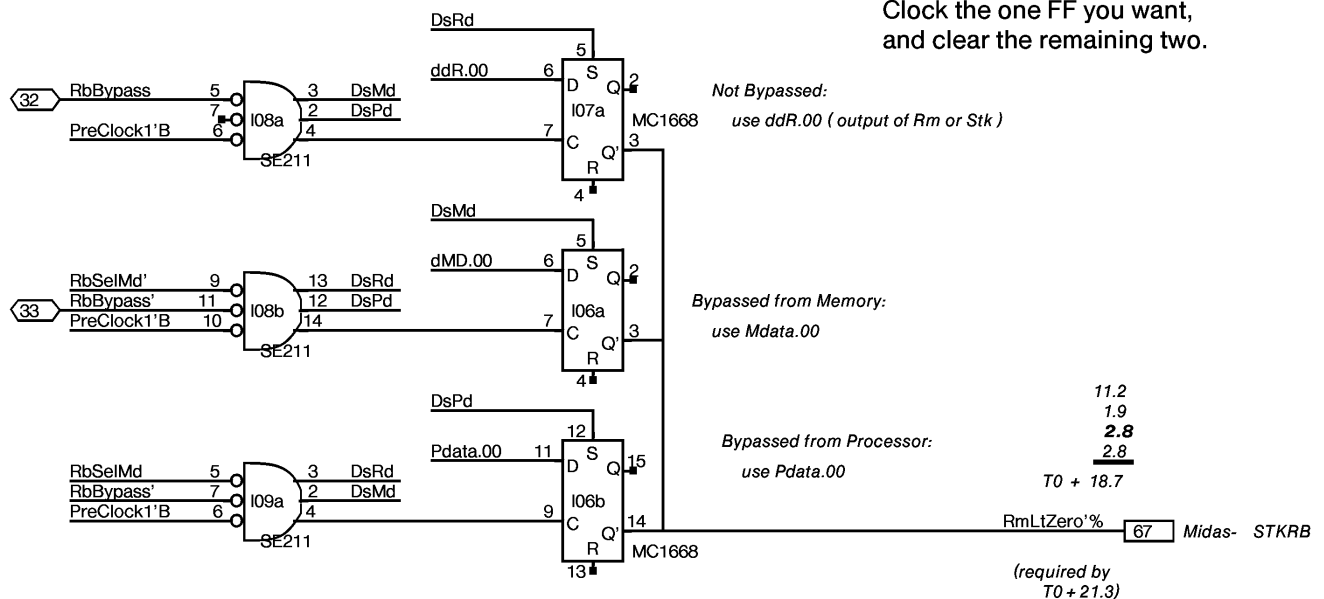
Arithmetic operation to data = 30.1

Arithmetic operation to carry = 28.0

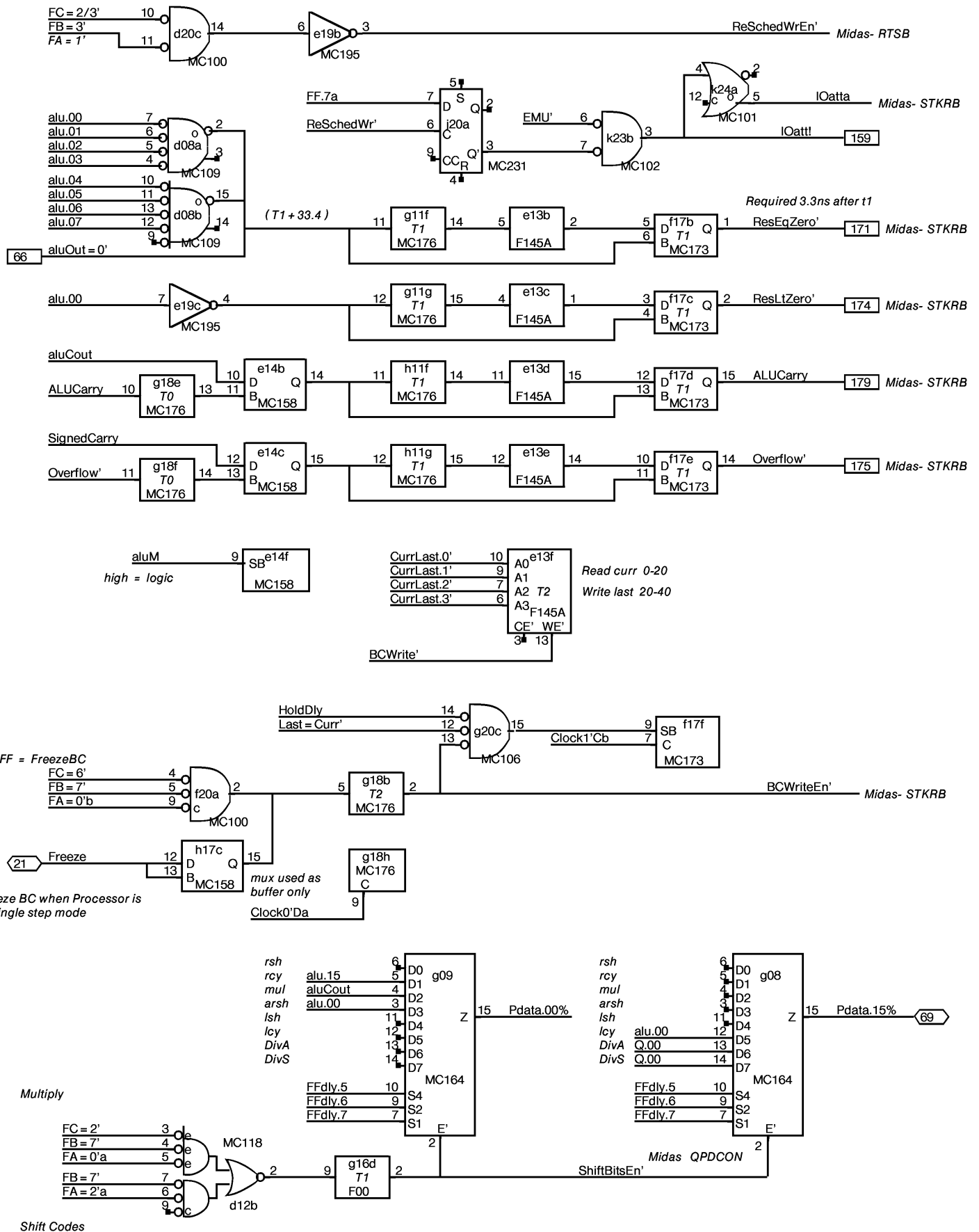


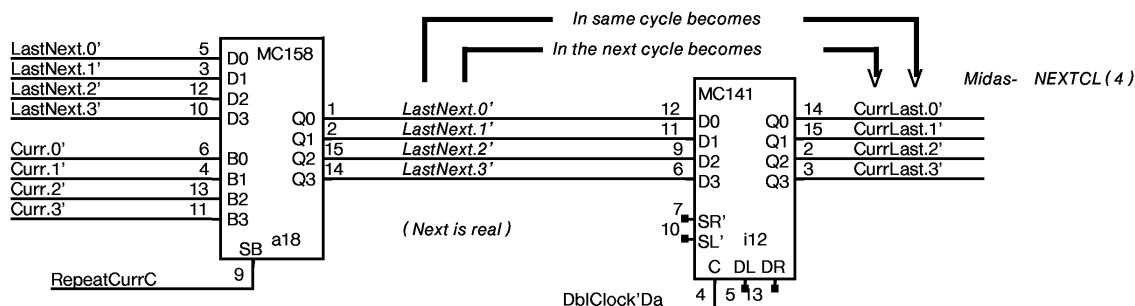
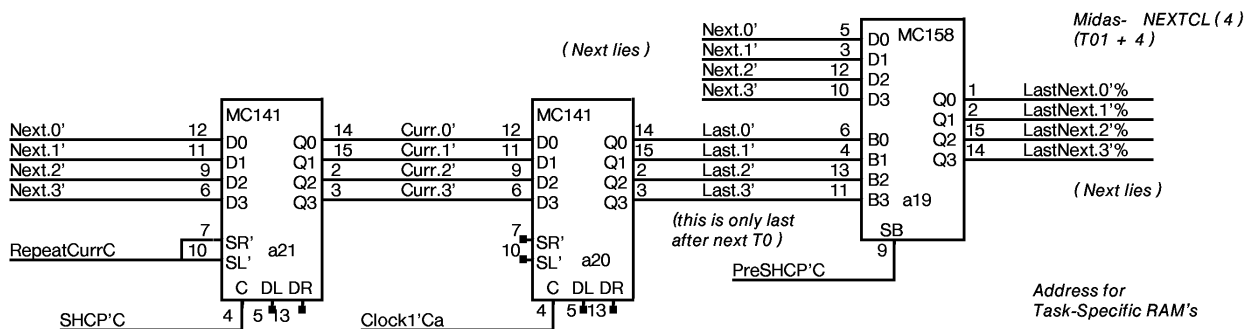
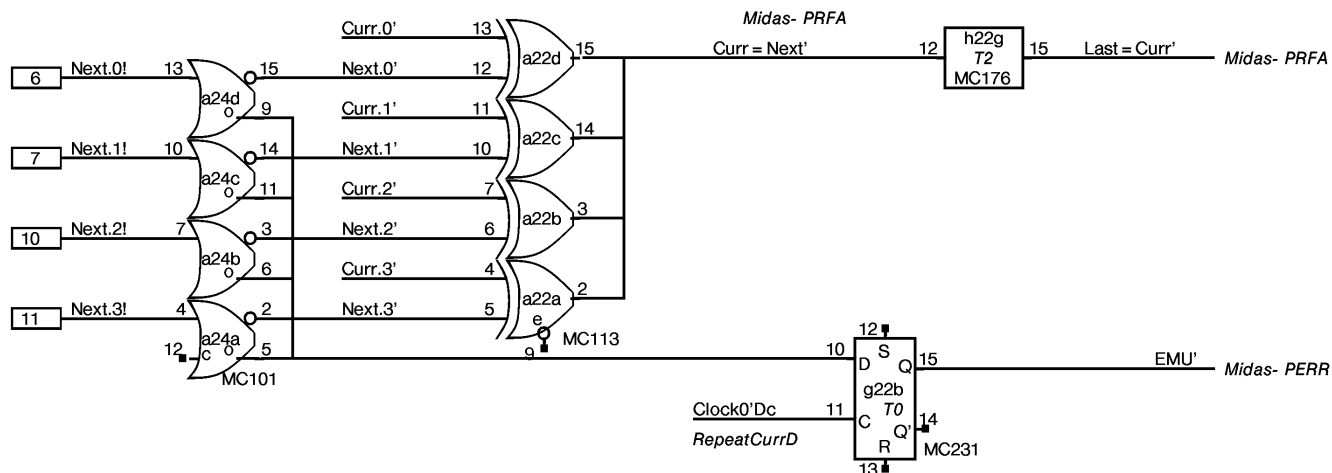
Fast Branch on Rm < 0

Clock the one FF you want,
and clear the remaining two.



This circuit will correctly
bypass R from Pdata or Mdata
When "RisIfData" is in effect, the fast branch
will be based on the contents of the addressed
RM or STK, bypassed if necessary.

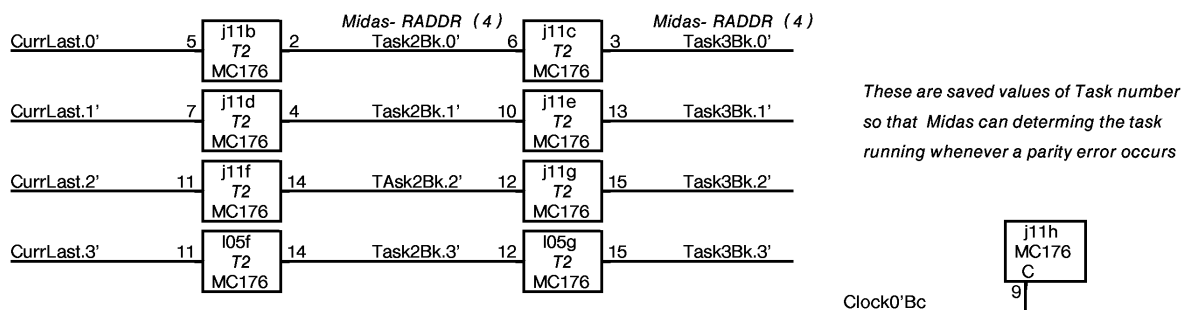


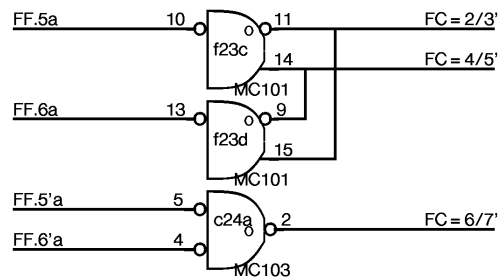
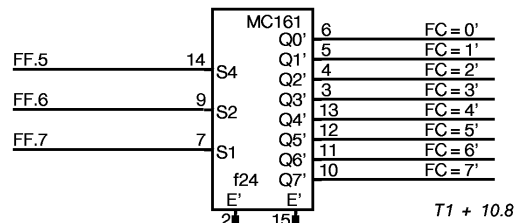
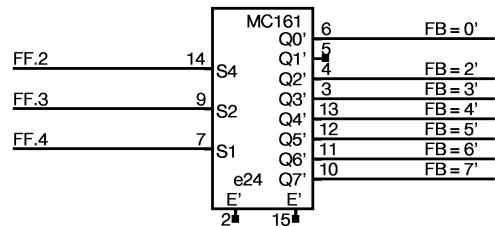
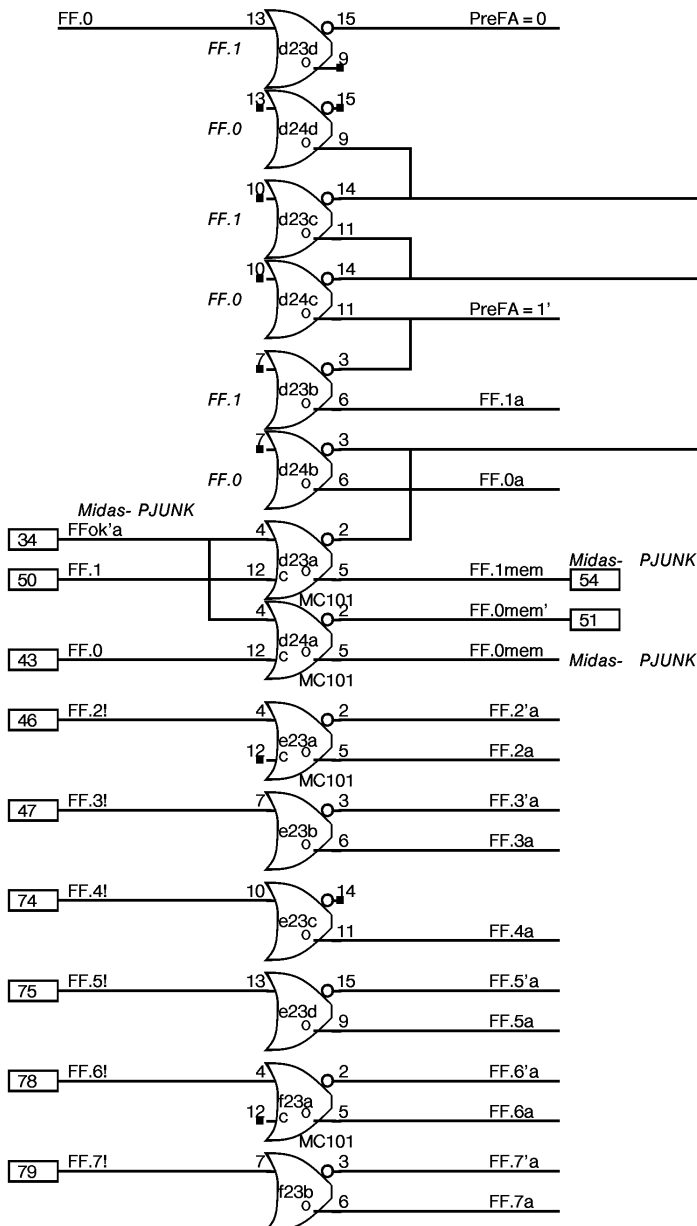
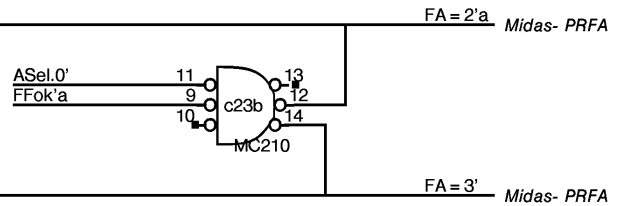
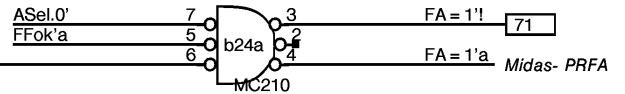
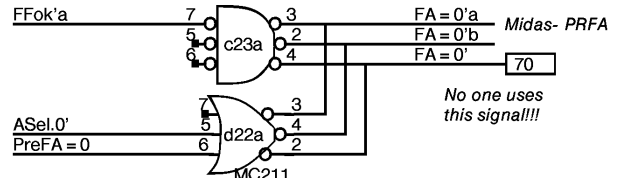
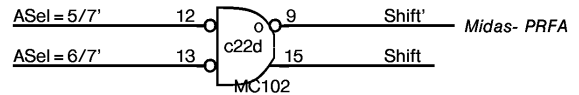
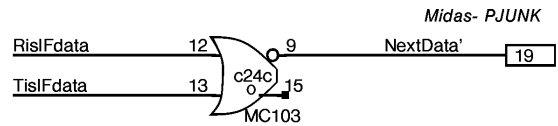
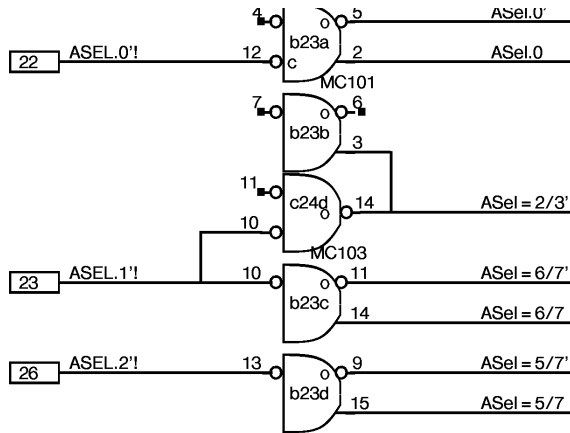


"RepeatCurrent" is asserted on certain occasions when the value on the NEXT bus may be invalid ("Next Lies") due to the combination of Block and Hold.

Task number tracking logic

Task number tracking logic (for Midas)





AMux decoding

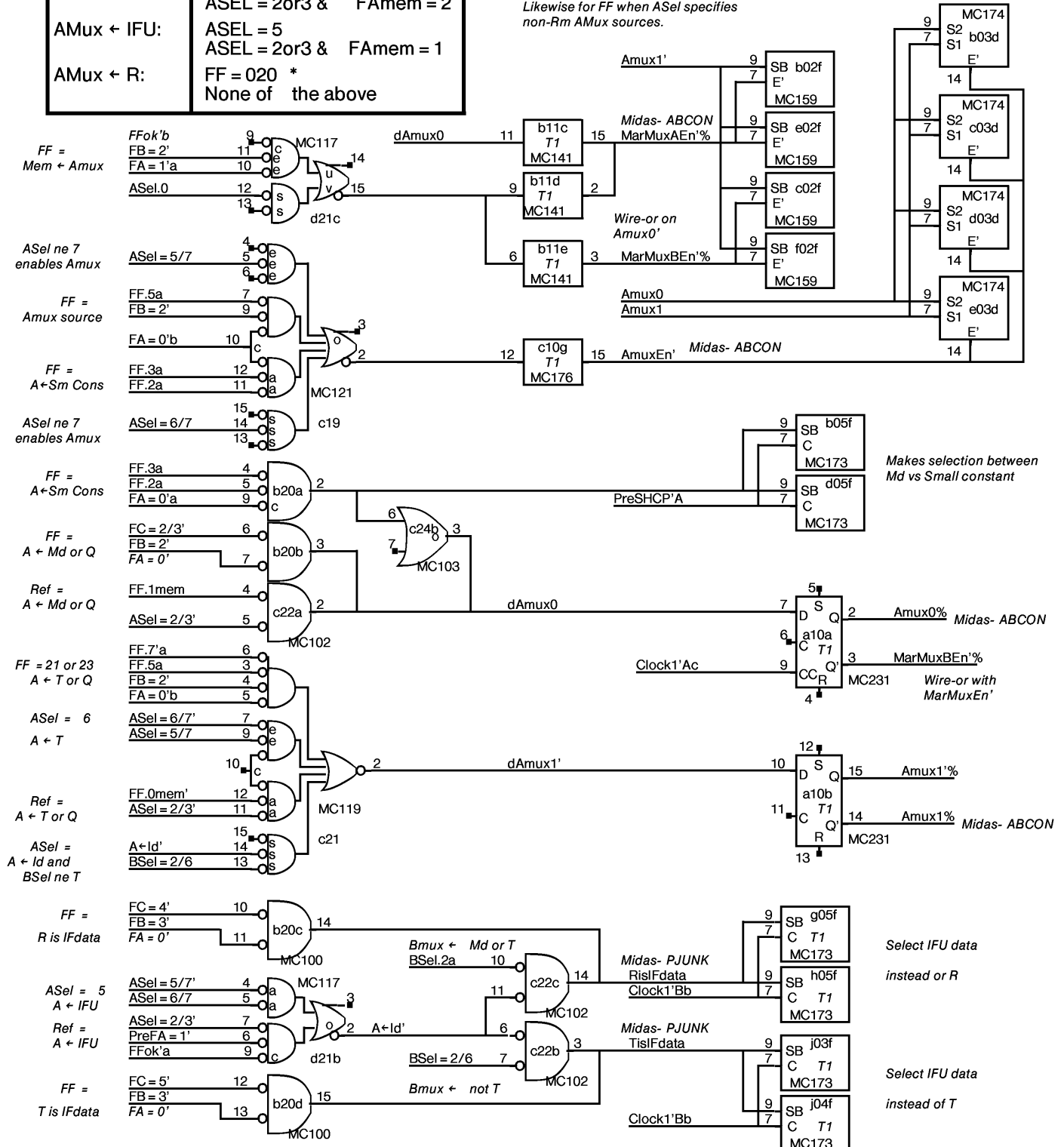
AMux ← FF:	FF = 0-17 *
AMux ← T:	FF = 021 * ASEL = 2or3 & FAmem = 3 ASEL = 6
AMux ← Md:	FF = 022 * ASEL = 2or3 & FAmem = 0
AMux ← Q:	FF = 023 * ASEL = 2or3 & FAmem = 2
AMux ← IFU:	ASEL = 5 ASEL = 2or3 & FAmem = 1
AMux ← R:	FF = 020 * None of the above

* The Amux is disabled by ASEL = 7 unless one of these codes are in effect

NOTE: ASEL selects and FF selects for the AMux are "OR'd" by this hardware. Thus ASEL codes selecting non-Rm sources of Amux must not be used when an FF specifies an ASEL source. Likewise for FF when ASEL specifies non-Rm AMux sources.

AMux encoding

Mux Input	Source
0	R or IFU data
1	T or IFU data
2	Md or Small Const.
3	Q

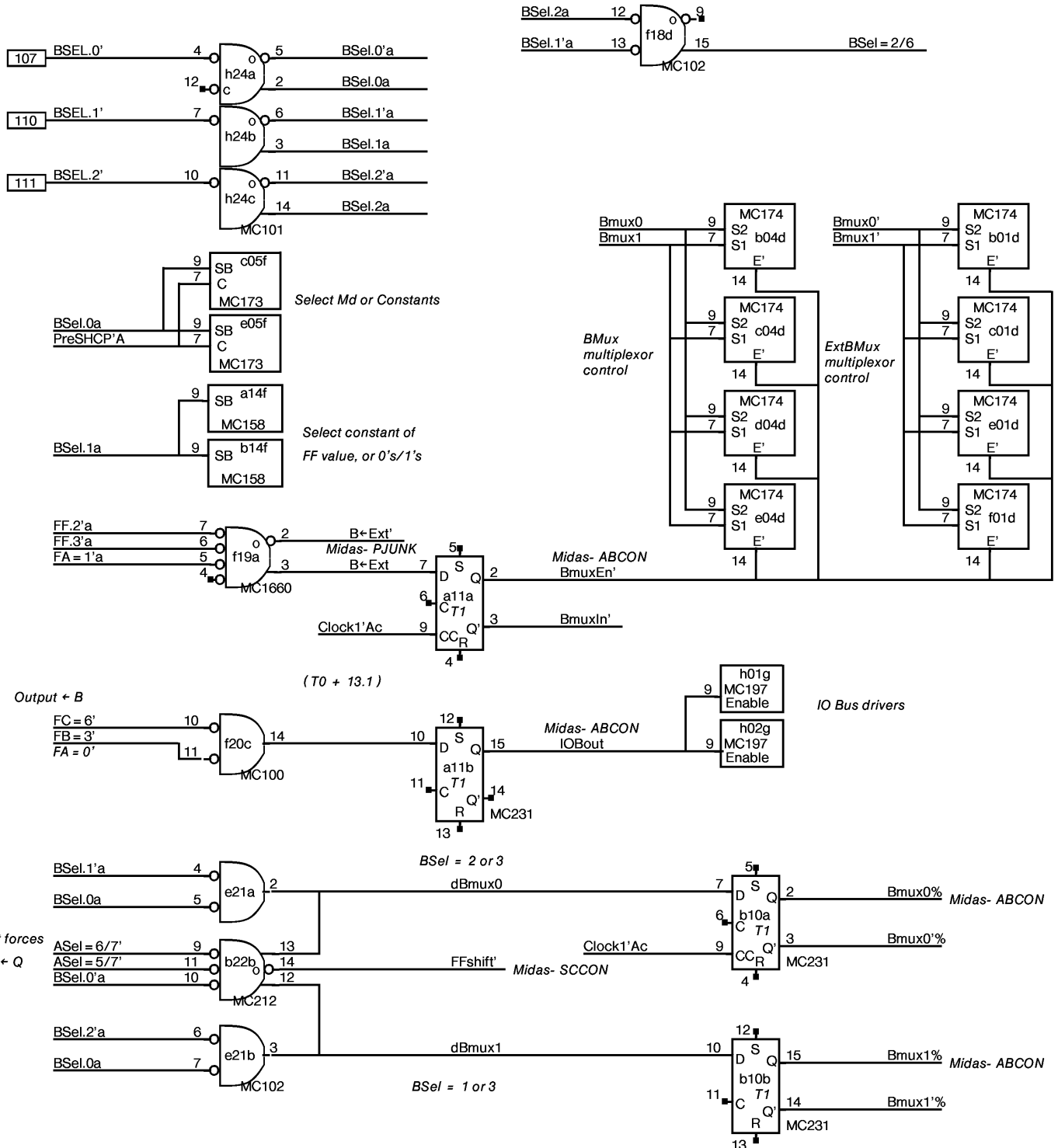


BSEL field decoding

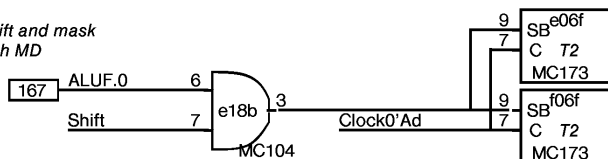
BSEL	INTERNAL	EXTERNAL
0	Md	--
1	R	--
2	T	Hold ← B
3	Q	Q ← B
4	0,,FF	--
5	377,,FF	--
6	FF,,0	--
7	FF,,377	--

BMux encoding

BMux	
0	Md or Constant
1	R
2	T
3	Q



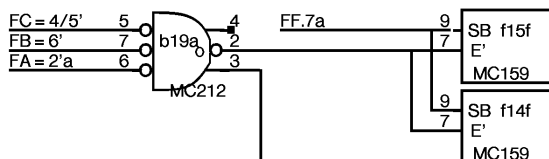
Shift and mask
with MD



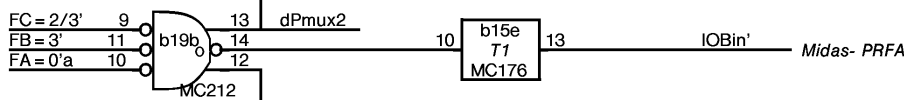
P Mux Encoding

Input	Source
0	ALU
1	0/Md, Cnt, Pointers
2	L shift
3	R shift
4	ALUFM
5	IO data
6	TIOA,,StkP
7	Shc

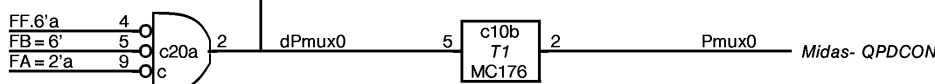
Pmux ←
RBase or Cnt



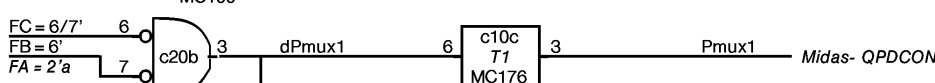
Pmux ← IOB



Pmux ← AluFm
Pmux ← Shc
Pmux ← StkP



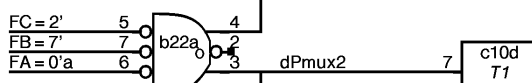
Pmux ← StkP
Pmux ← Shc



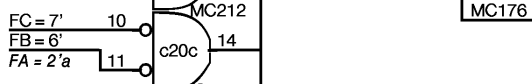
Pmux ←
Pmux Lshift 1
Pmux rshift 1



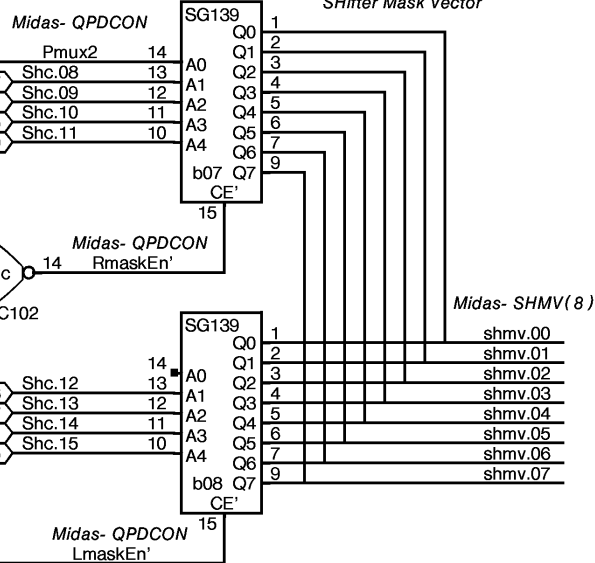
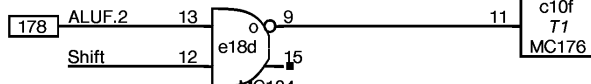
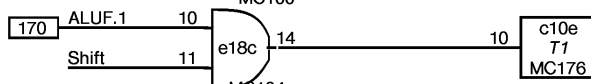
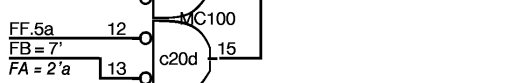
FF = Multiply
Pmux ←
Pmux rshift 1



Pmux ← Shc



Pmux ←
Pmux rshift 1



Lmask

Shc.12-15

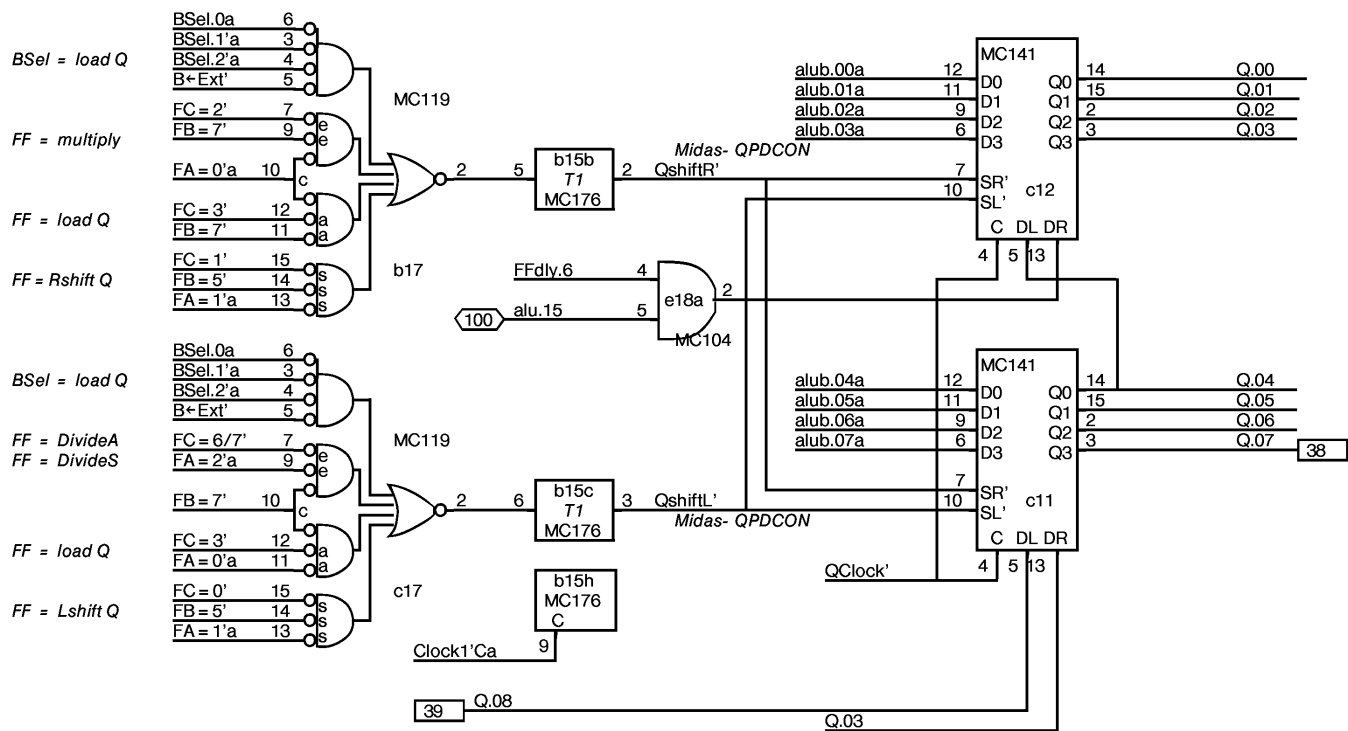
	00	01	02	03	04	05	06	07
0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0
3	1	1	1	0	0	0	0	0
4	1	1	1	1	0	0	0	0
5	1	1	1	1	1	0	0	0
6	1	1	1	1	1	1	0	0
7	1	1	1	1	1	1	1	0
10	1	1	1	1	1	1	1	1
11	1	1	1	1	1	1	1	1
12	1	1	1	1	1	1	1	1
13	1	1	1	1	1	1	1	1
14	1	1	1	1	1	1	1	1
15	1	1	1	1	1	1	1	1
16	1	1	1	1	1	1	1	1
17	1	1	1	1	1	1	1	1
20- 37	1	1	1	1	1	1	1	1

Rmask

Shc.08-11

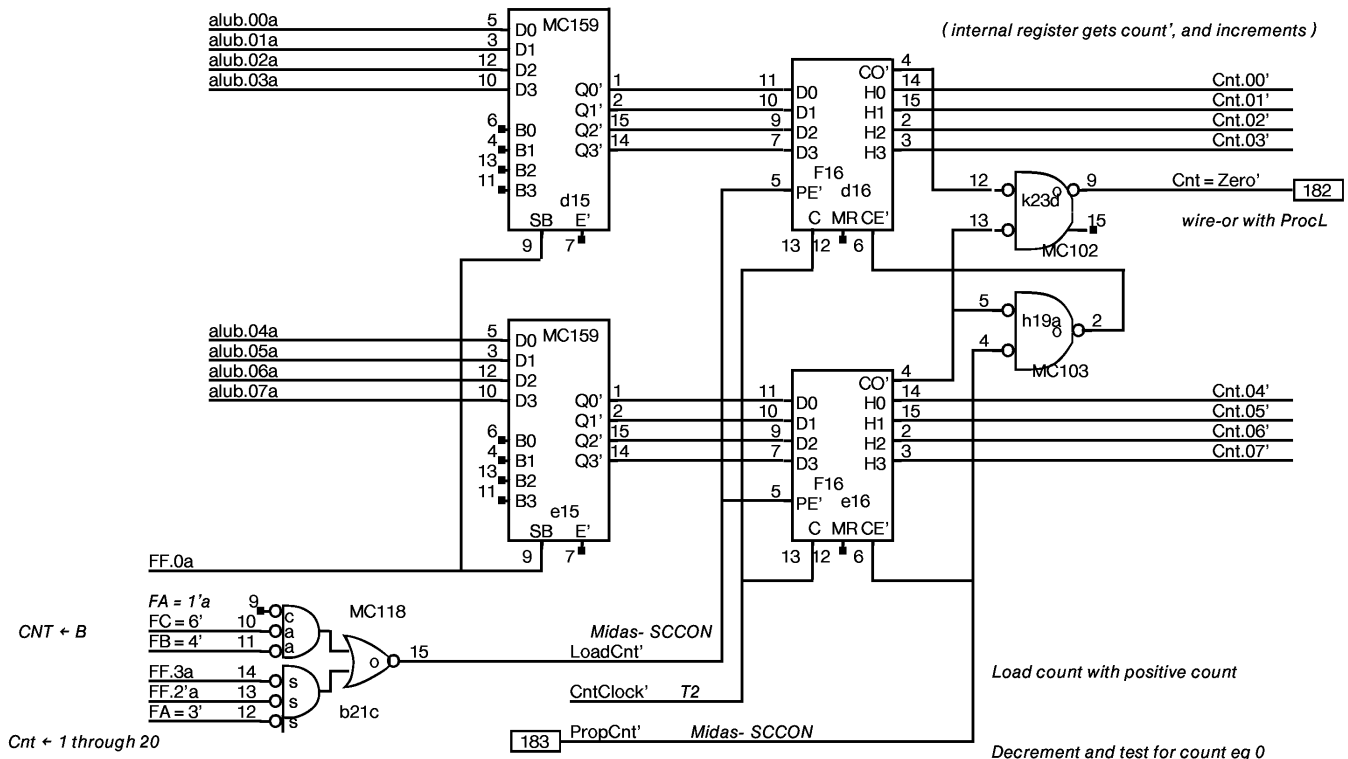
	00	01	02	03	04	05	06	07
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	1
12	0	0	0	0	0	0	1	1
13	0	0	0	0	0	1	1	1
14	0	0	0	0	1	1	1	1
15	0	0	0	1	1	1	1	1
16	0	0	1	1	1	1	1	1
17	0	1	1	1	1	1	1	1
20- 37	1	1	1	1	1	1	1	1

NOTE: The prom patterns are designed so that a one into address bit 0 will produce all 1's on the output. This allows the odd address inputs to Pmux to be selected.

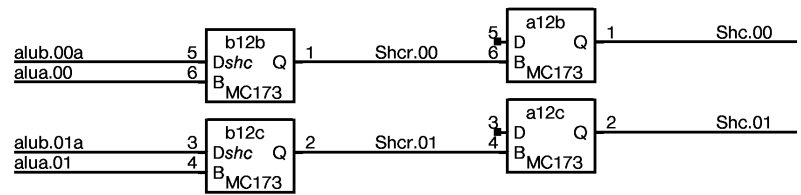


Q Register

Count Register



Extra Bits



R/T Select

	Shc.02	Shc.03	ProcH ShA ShB	ProcL ShA ShB
R,,R	0	0	R R	R R
R,,T	0	1	R T	T R
T,,R	1	0	T R	R T
T,,T	1	1	T T	T T

Note R selected with "0"
T selected with "1"

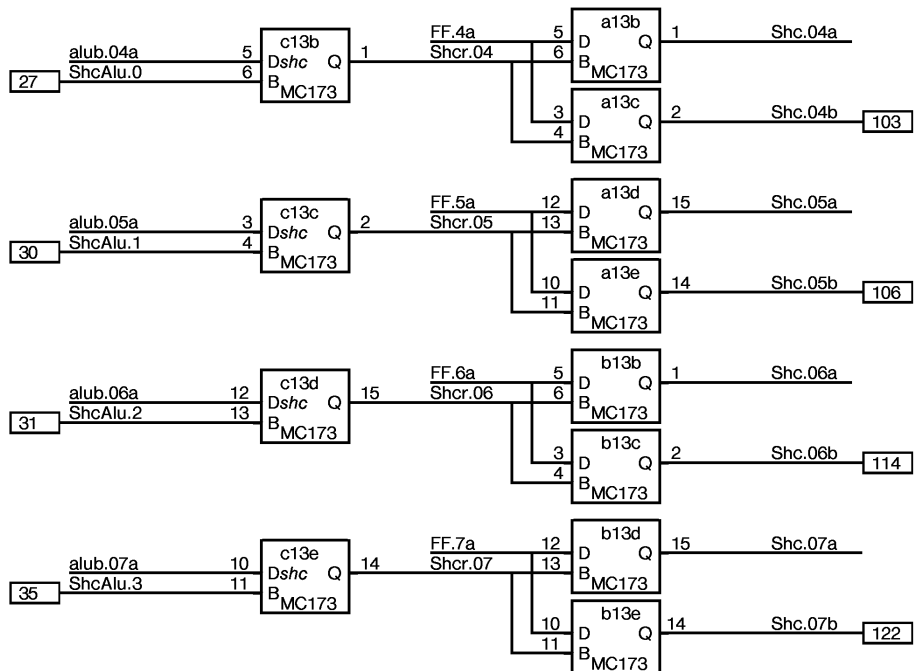


IntBmux
RF / WF

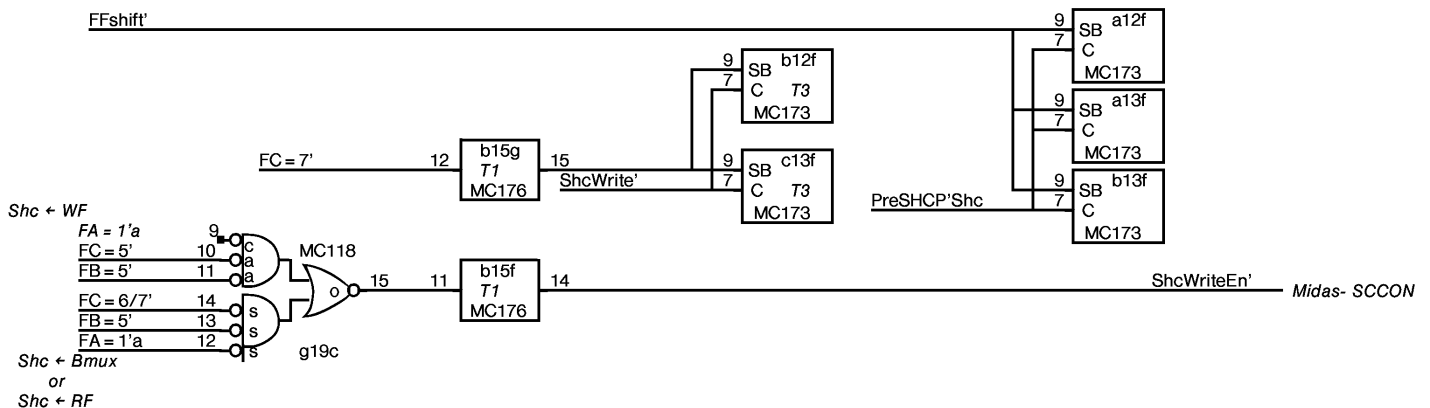
FF

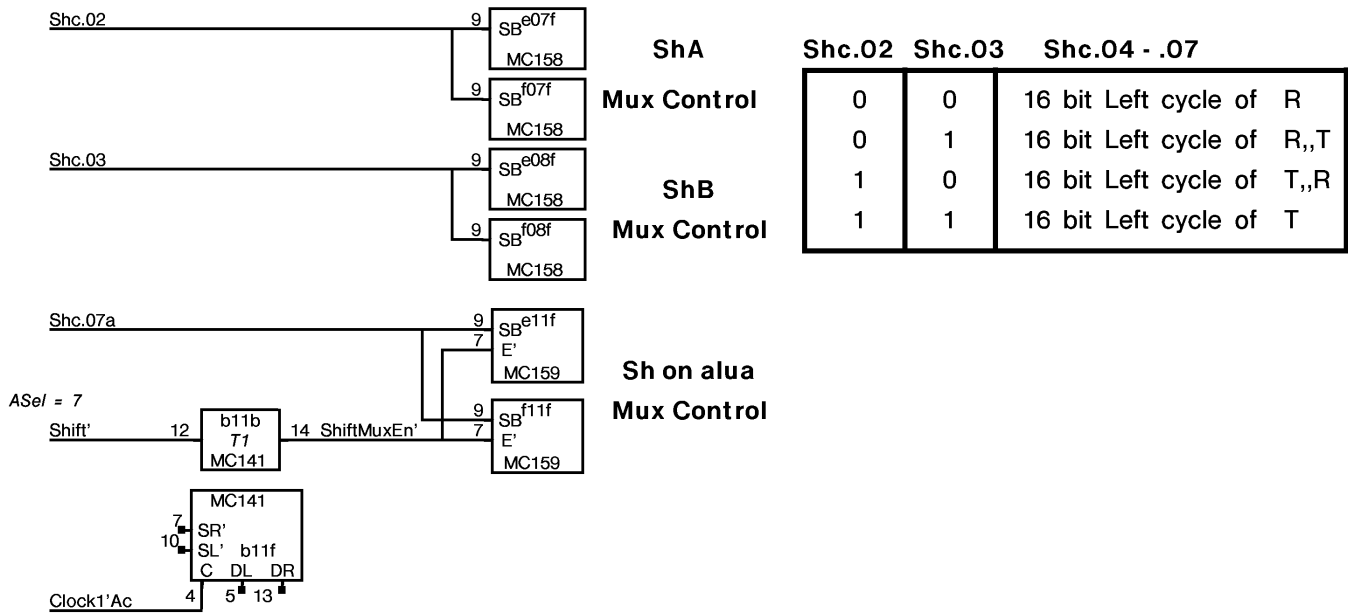
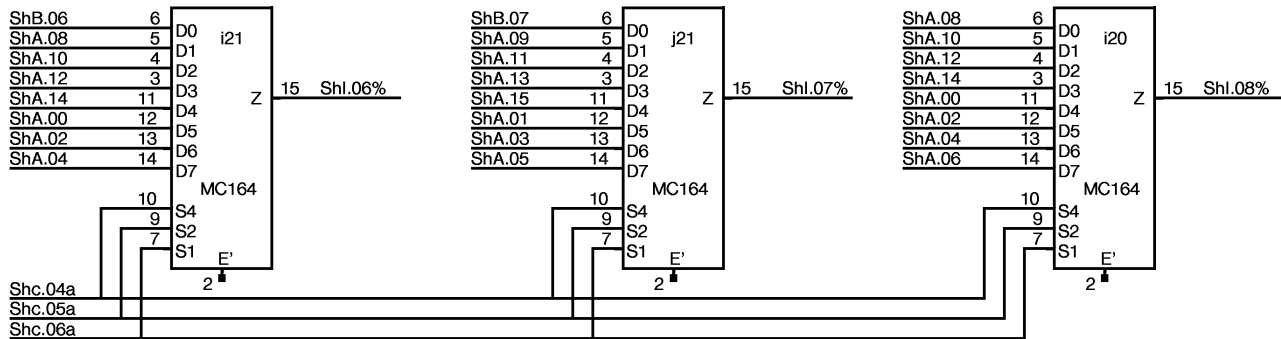
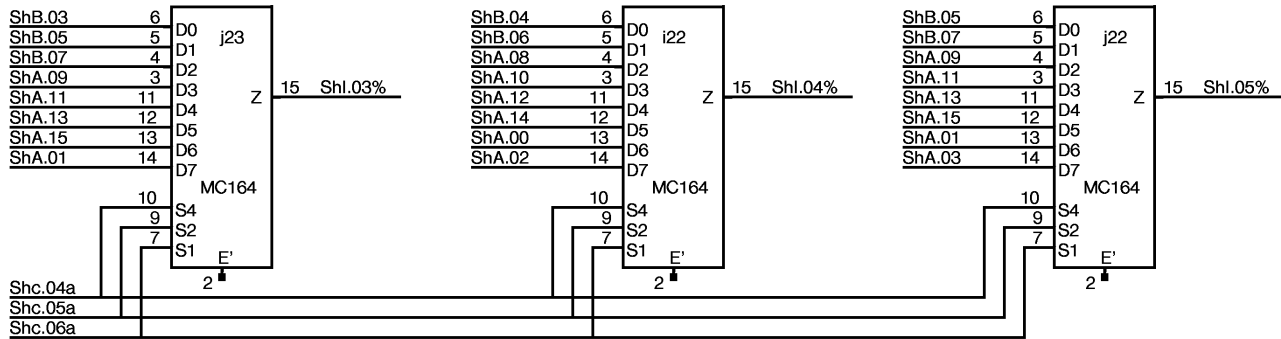
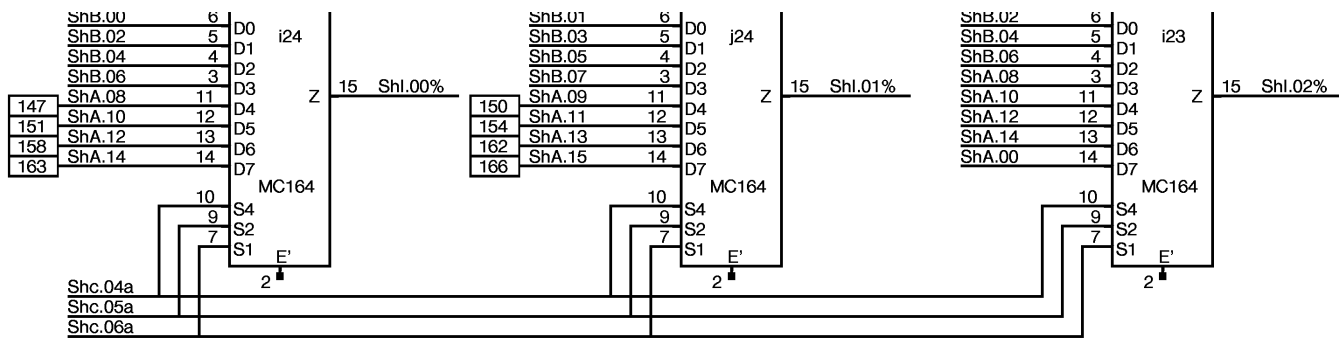
Shift control

Shift Count



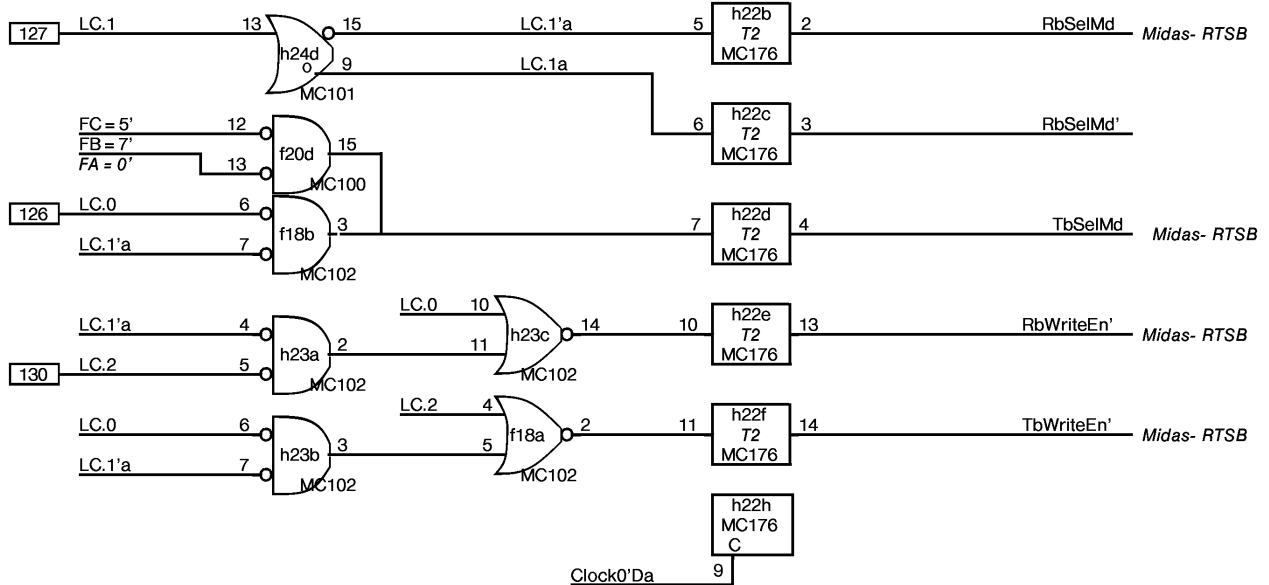
Done this way so FF shifts
do not use Shc - IO tasks can
use the shifter without saving
and restoring Shc.



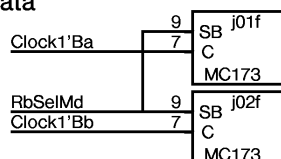


LC			R	T
0	1	2		
0	0	0	-	-
0	0	1	-	Pd *
0	1	0	Pd	Md
0	1	1	-	Md
1	0	0	Md	-
1	0	1	Md	Pd *
1	1	0	Pd	-
1	1	1	Pd	Pd *

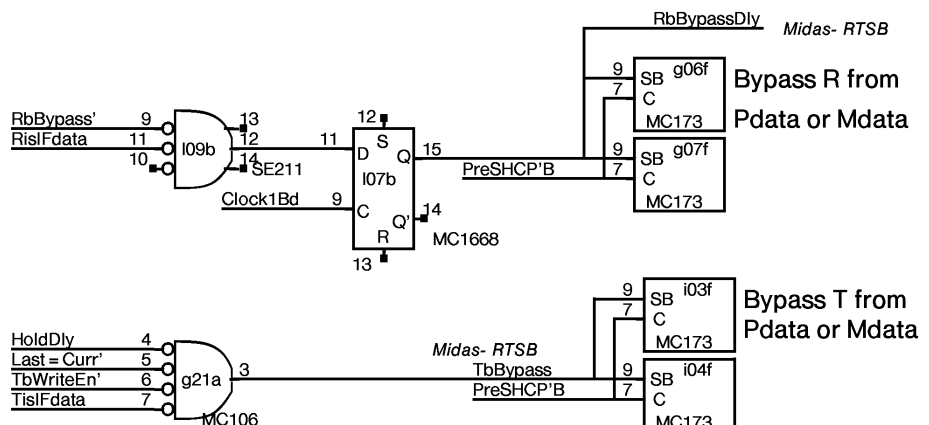
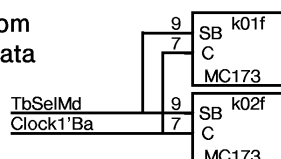
* Md if used when
FF = 075

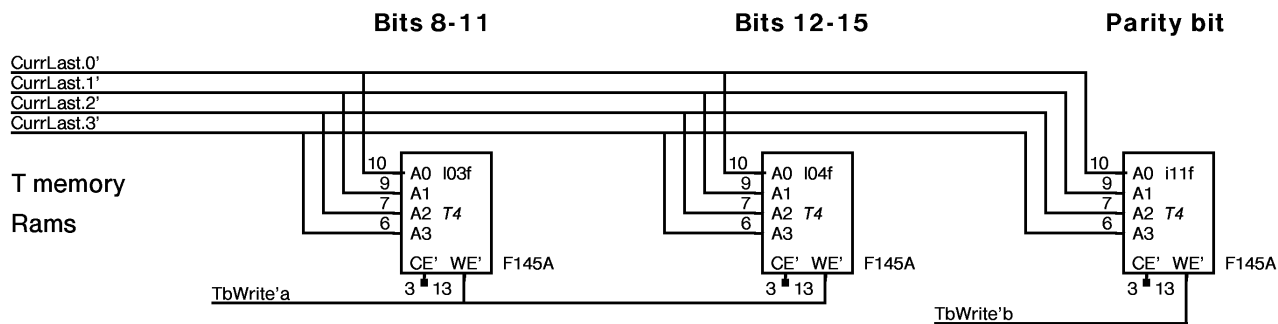
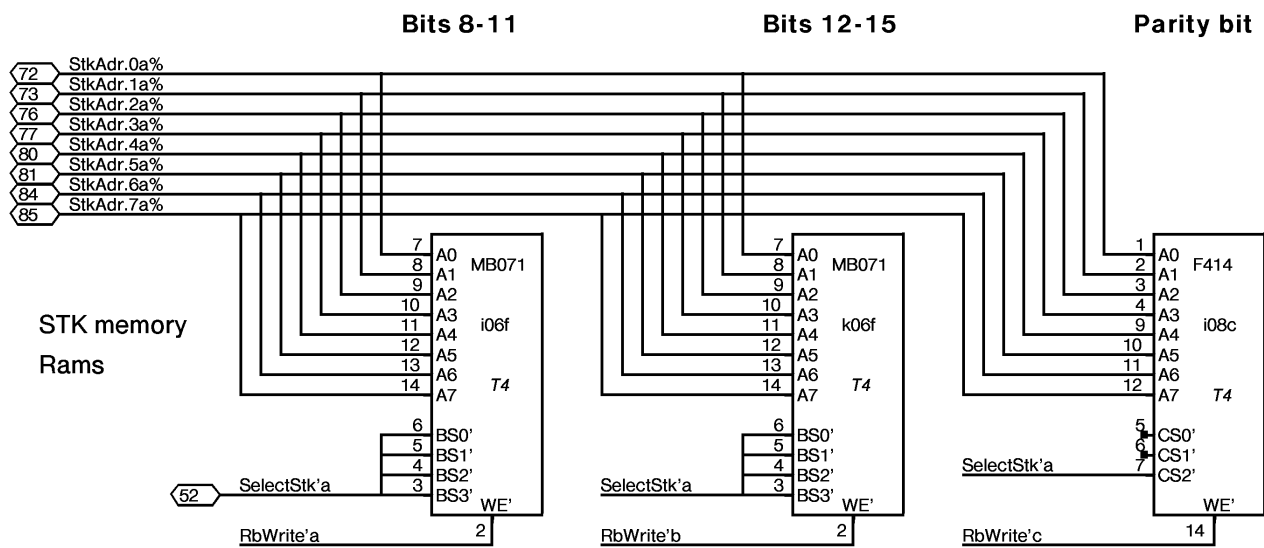
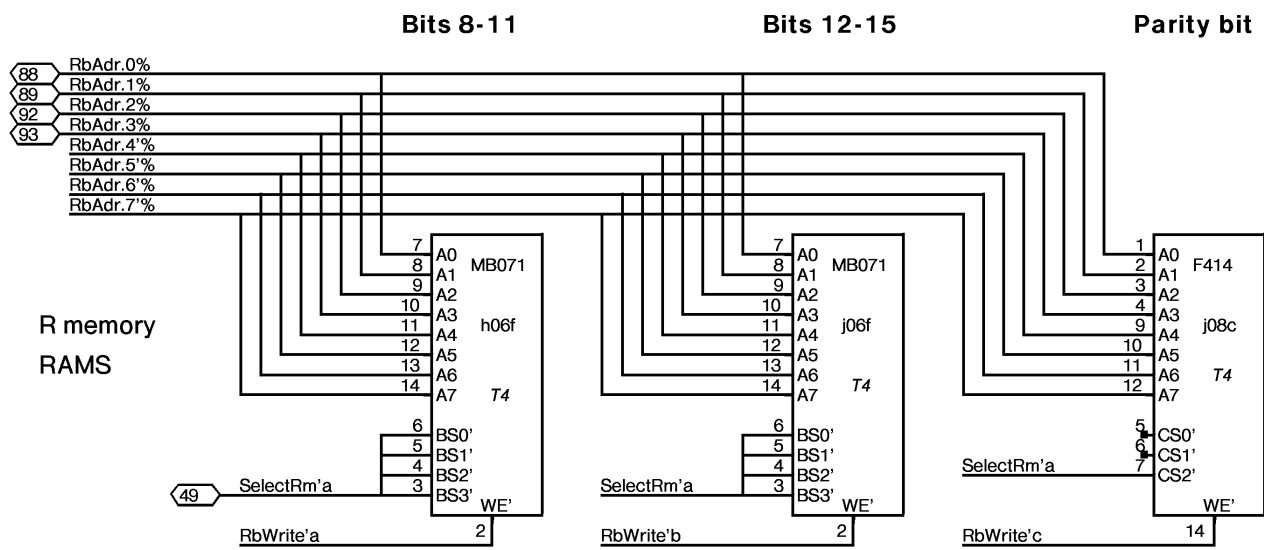


Write R From
Pdata or Mdata

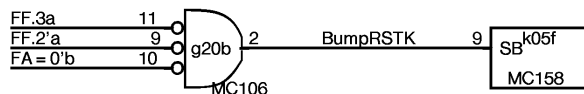


Write T From
Pdata or Mdata



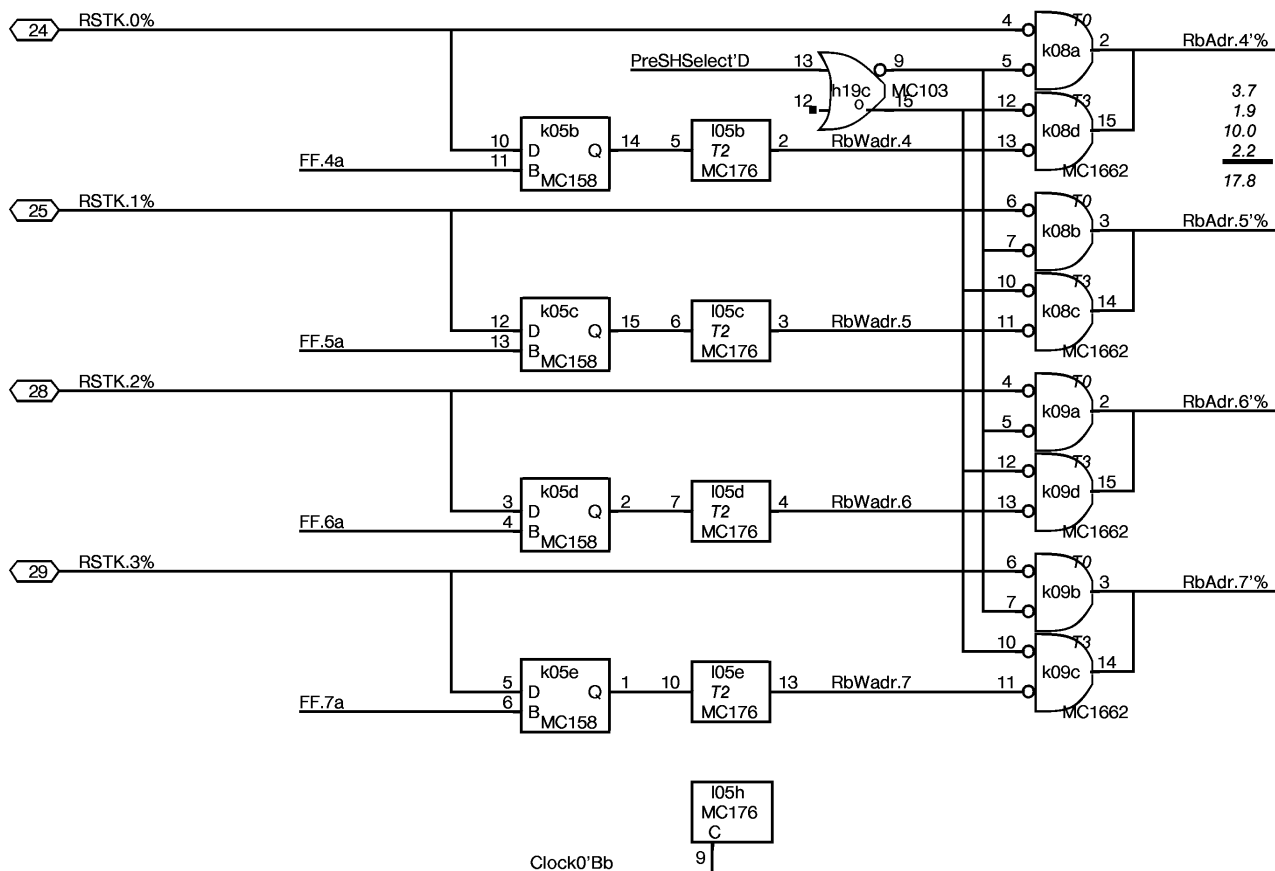


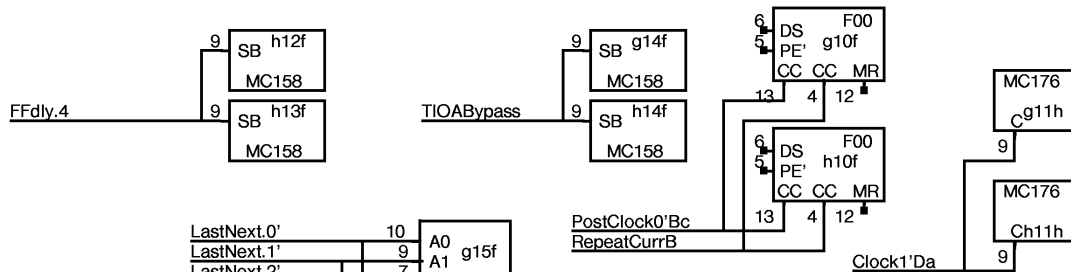
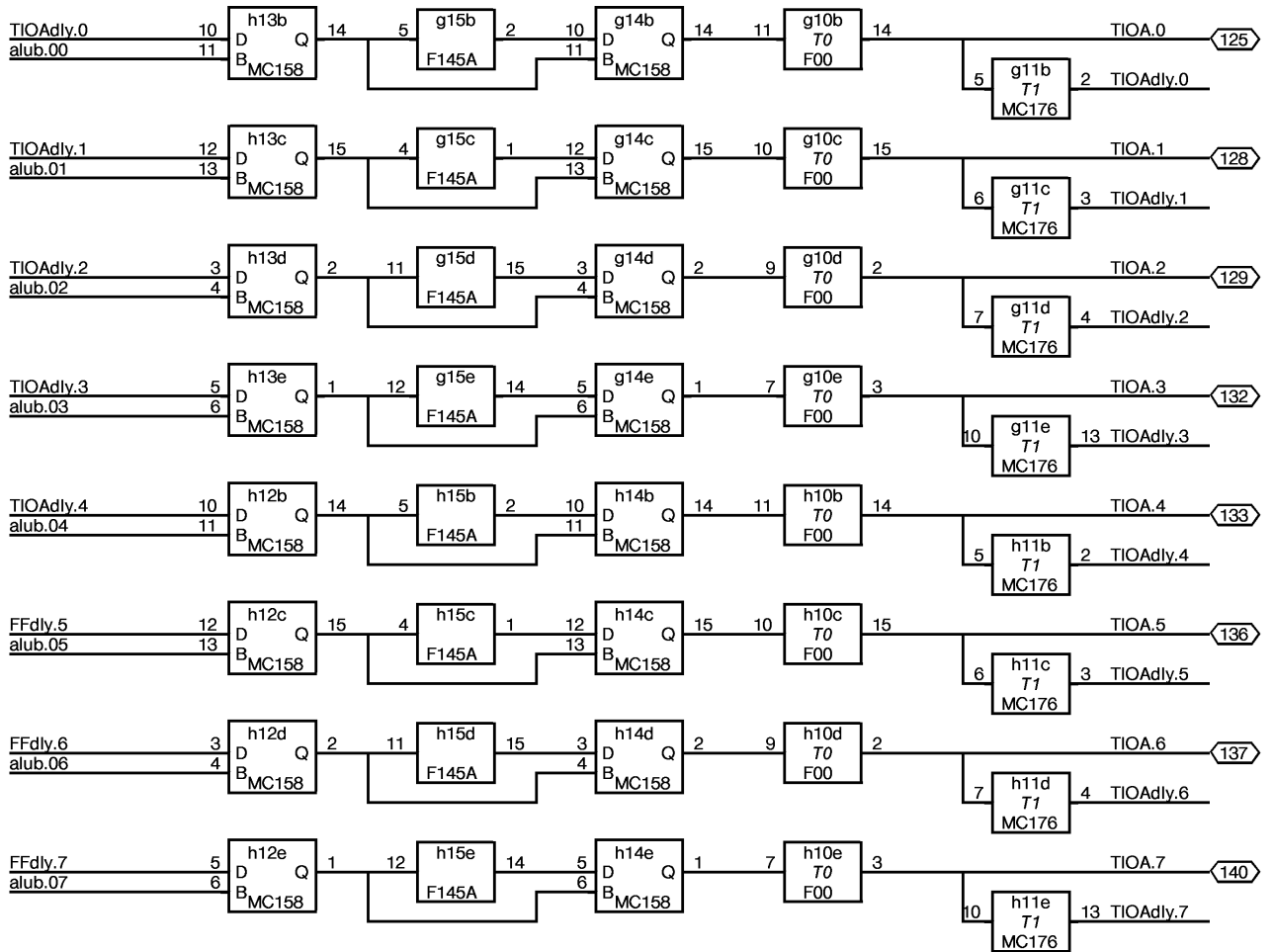
RbWadr ← RBase,,FF



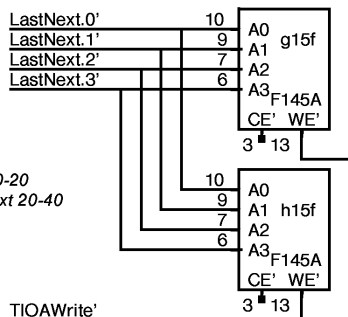
RSTK Control for bits 4 - 7 of the Rm address

RSTK bits that make up Rm address bits 4-7



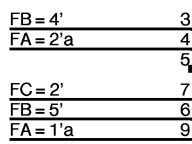


Write to Last 0-20
Read from Next 20-40



FF =
TIOA = TIOA,,FF

TIOAWrite'

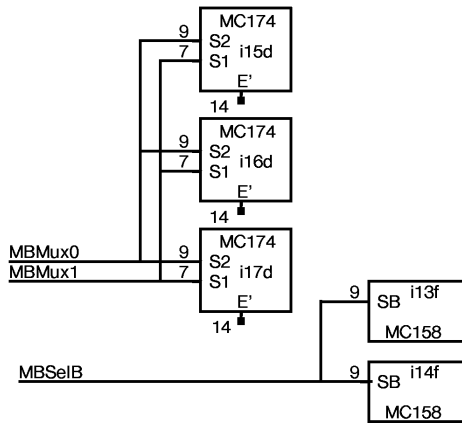


FF =
TIOA + Bmux

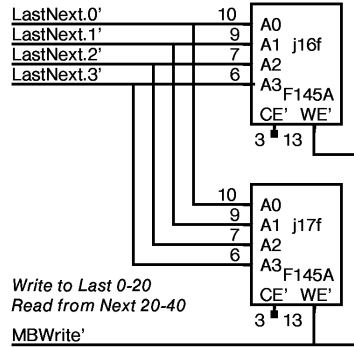
Curr = Next'
Hold

TIOAWriteEn' Midas- ALUCON

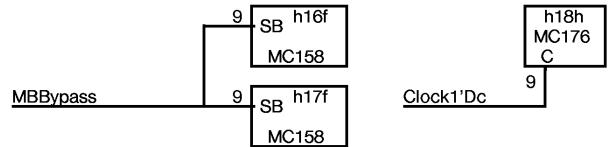
TIOABypass% Midas- ALUCON



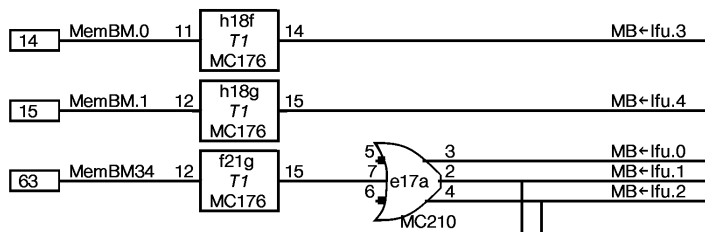
These select between BMux and "other" source to the MemBase RAM, and are always selected.



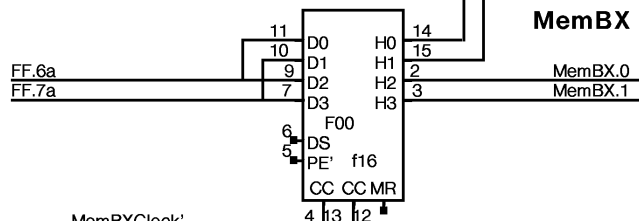
Write to Last 0-20
Read from Next 20-40
MBWrite'



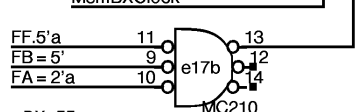
These select between task specific RAM (normal) and the input to the RAM for Bypassing



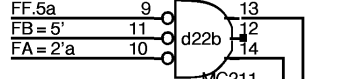
MemBX register



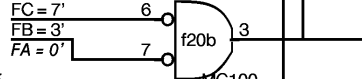
MemBX + FF



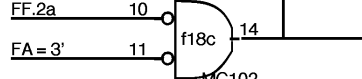
MemBase + MemBX,,FF



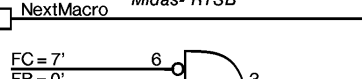
MemBase + MemBase XOR 1



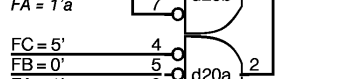
MemBase + FF



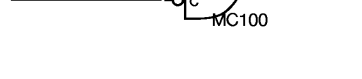
MemBase + MemBX,,MemBM NextMacro



MemBase + B



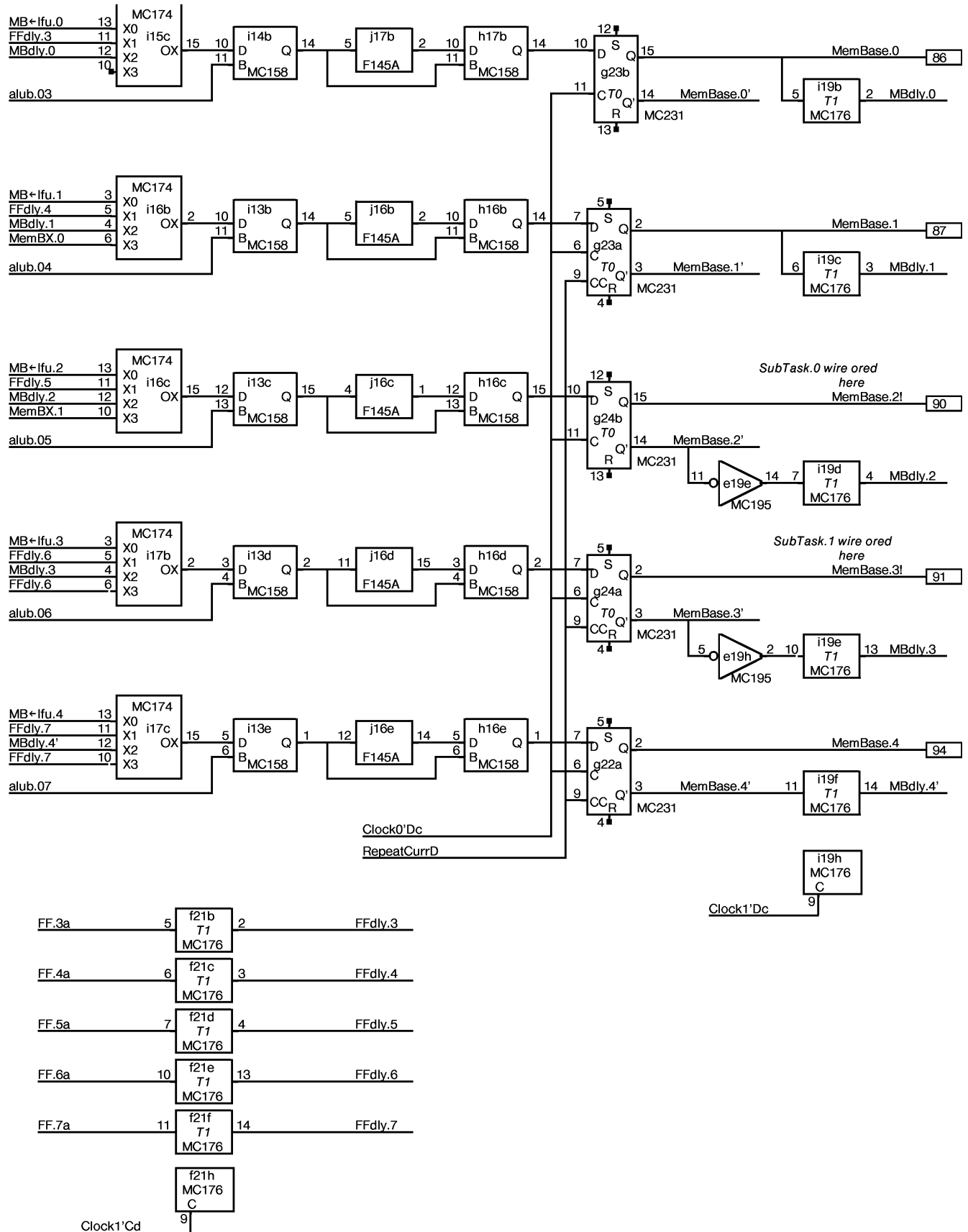
MemBase + B

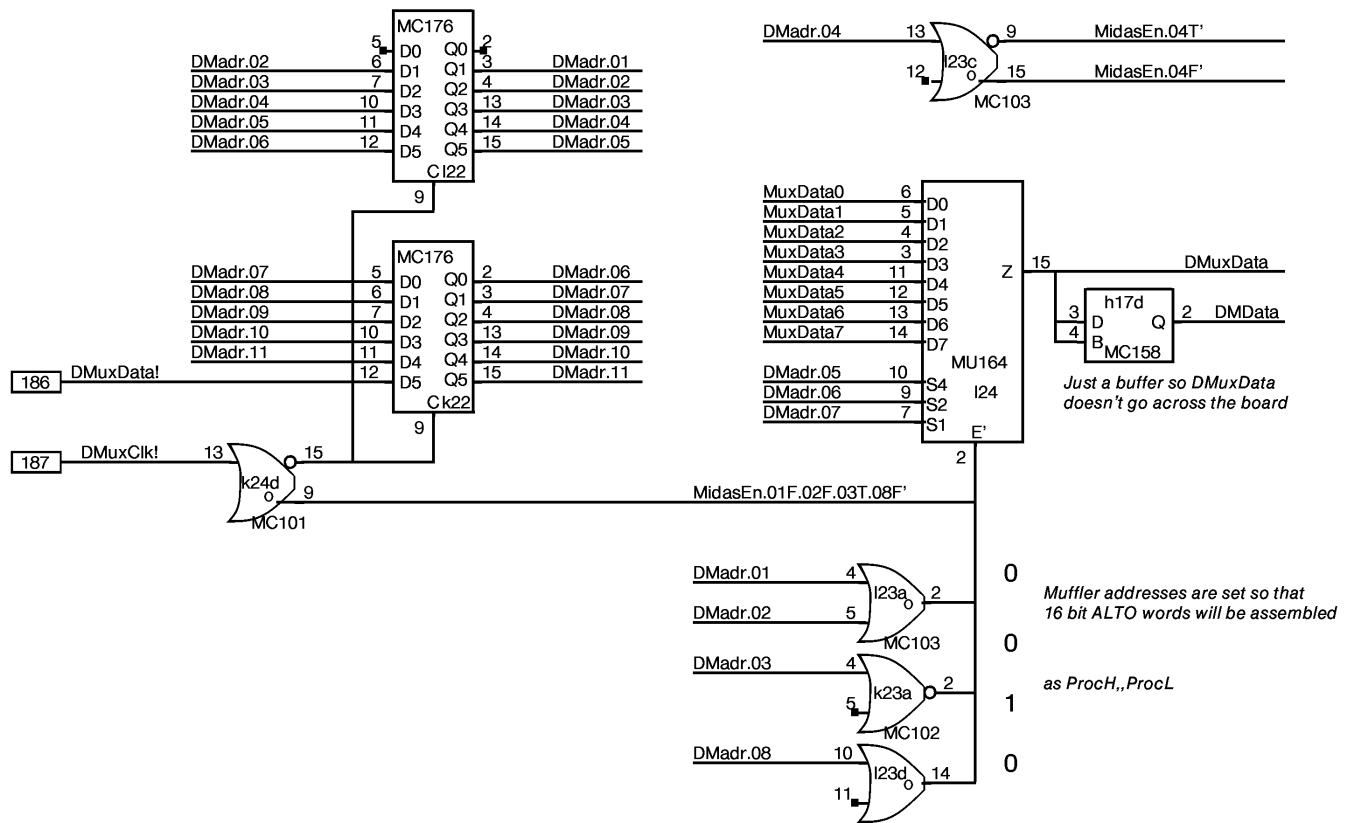


Mux encoding

MC174	0	MemBX,,lfuMB
	1	FF value
	2	XOR of current MemBX,,FF
MC158	3	BMux

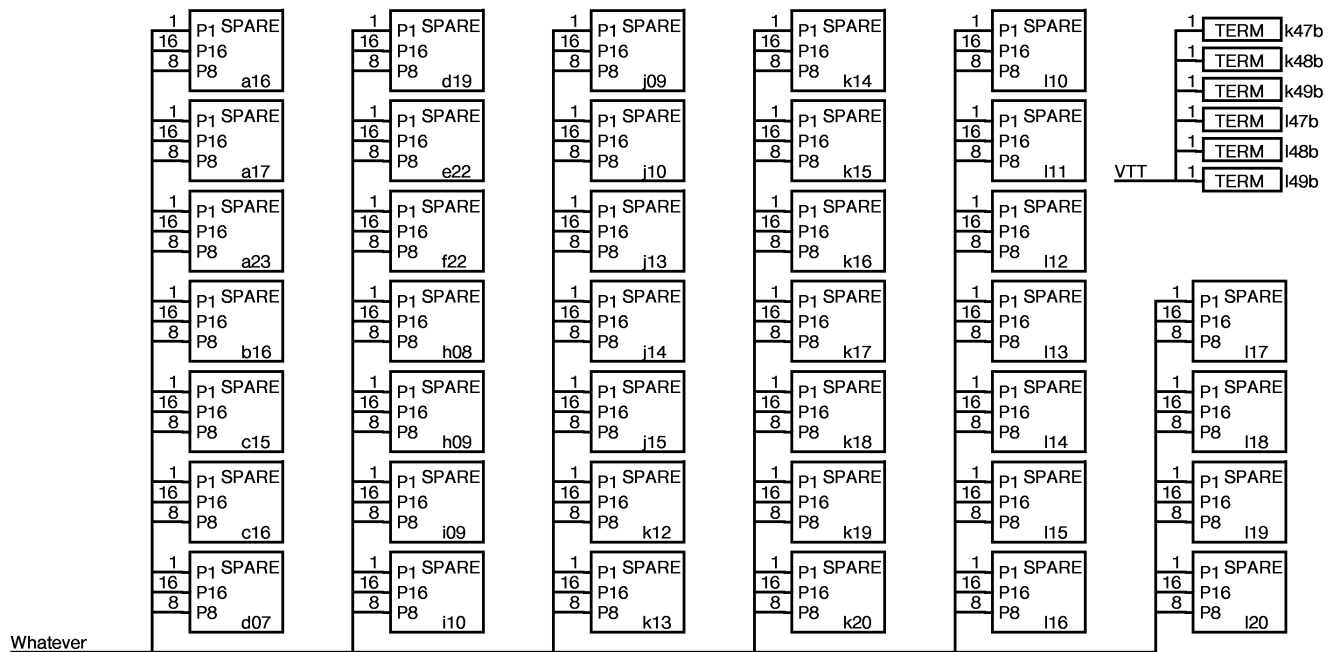
Curr = Next'
Hold



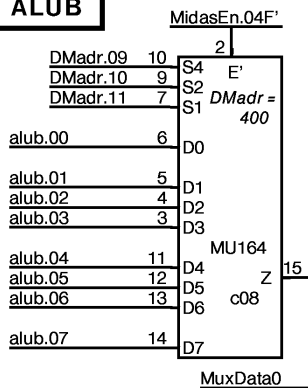
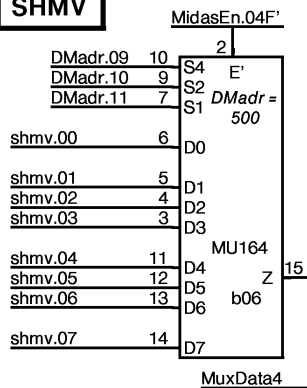
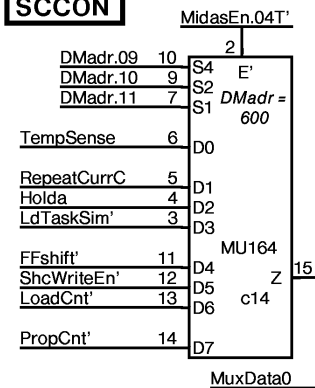
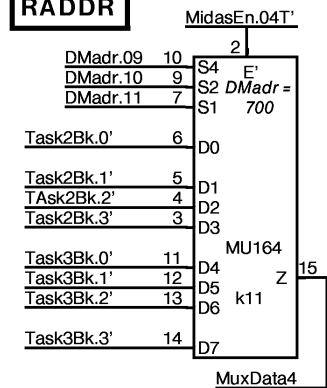
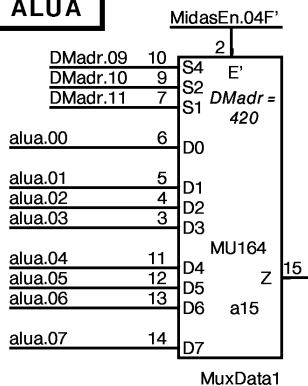
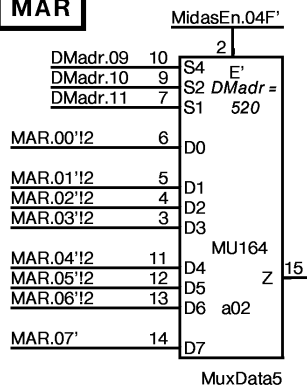
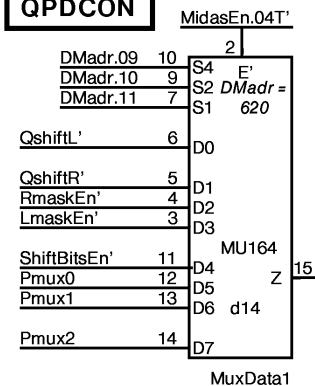
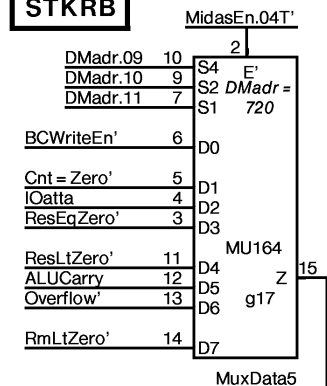
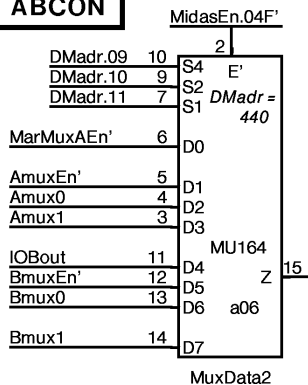
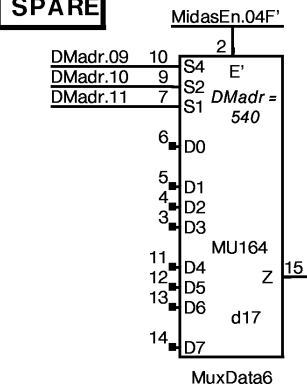
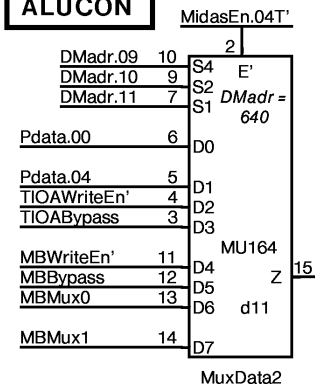
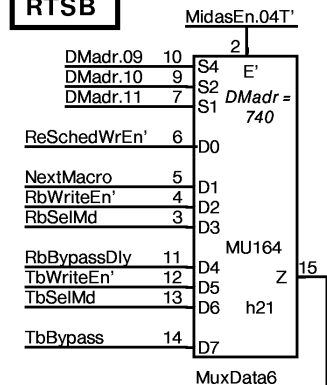
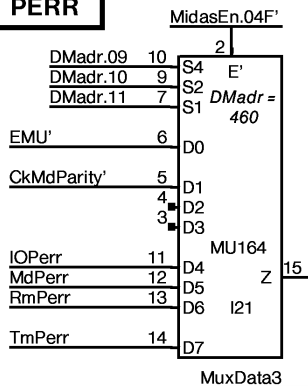
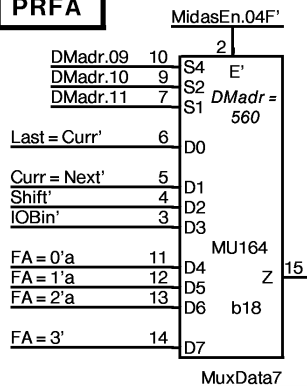
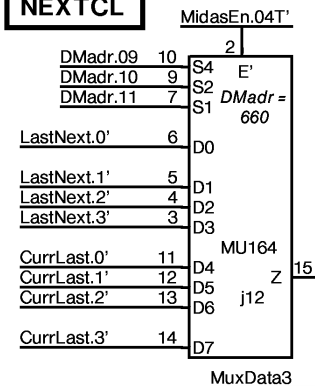
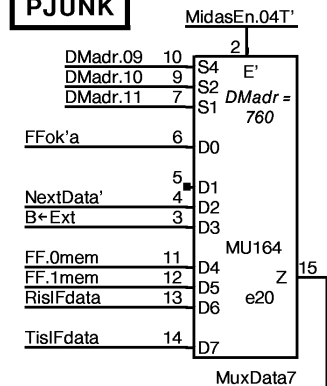


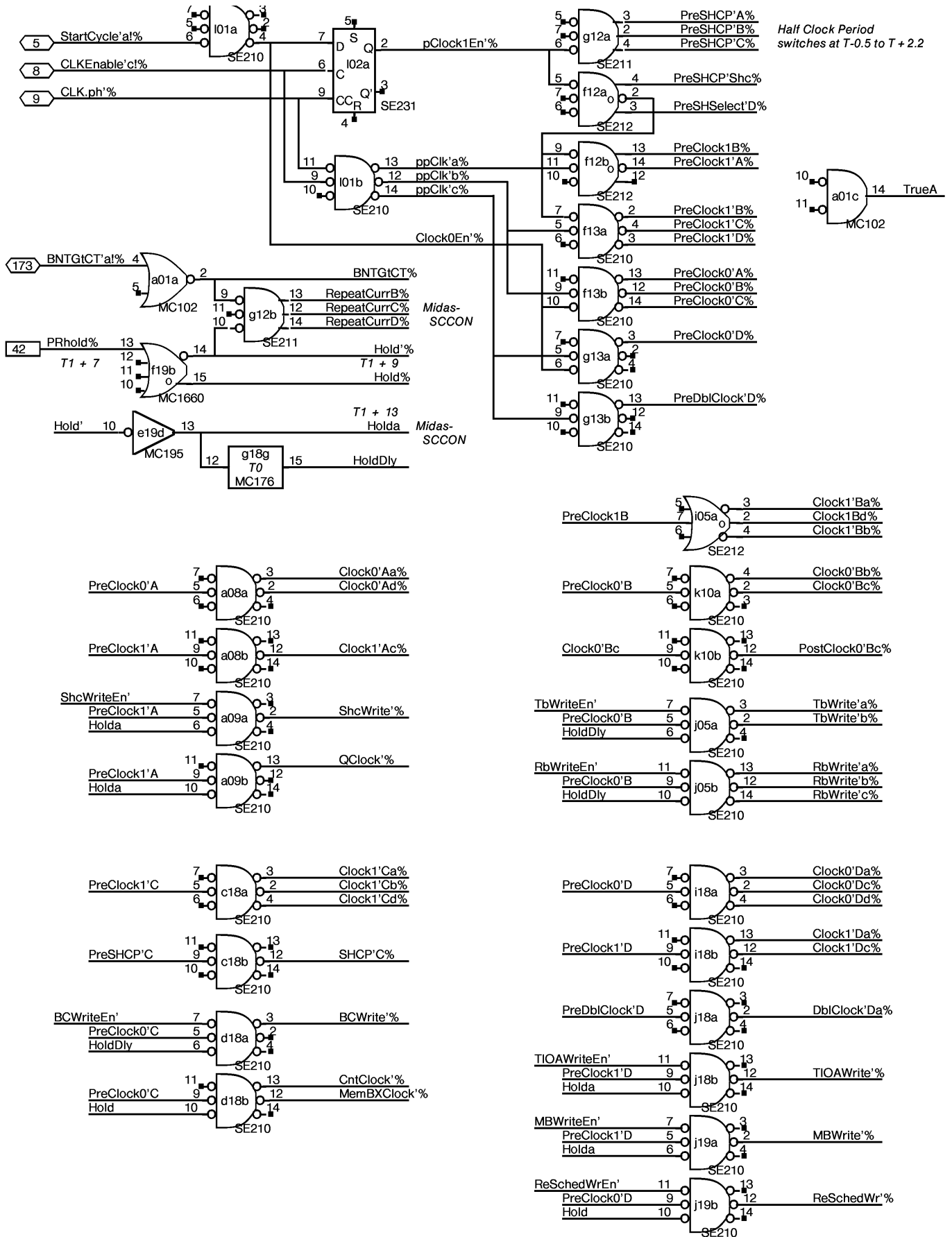
Midas Control

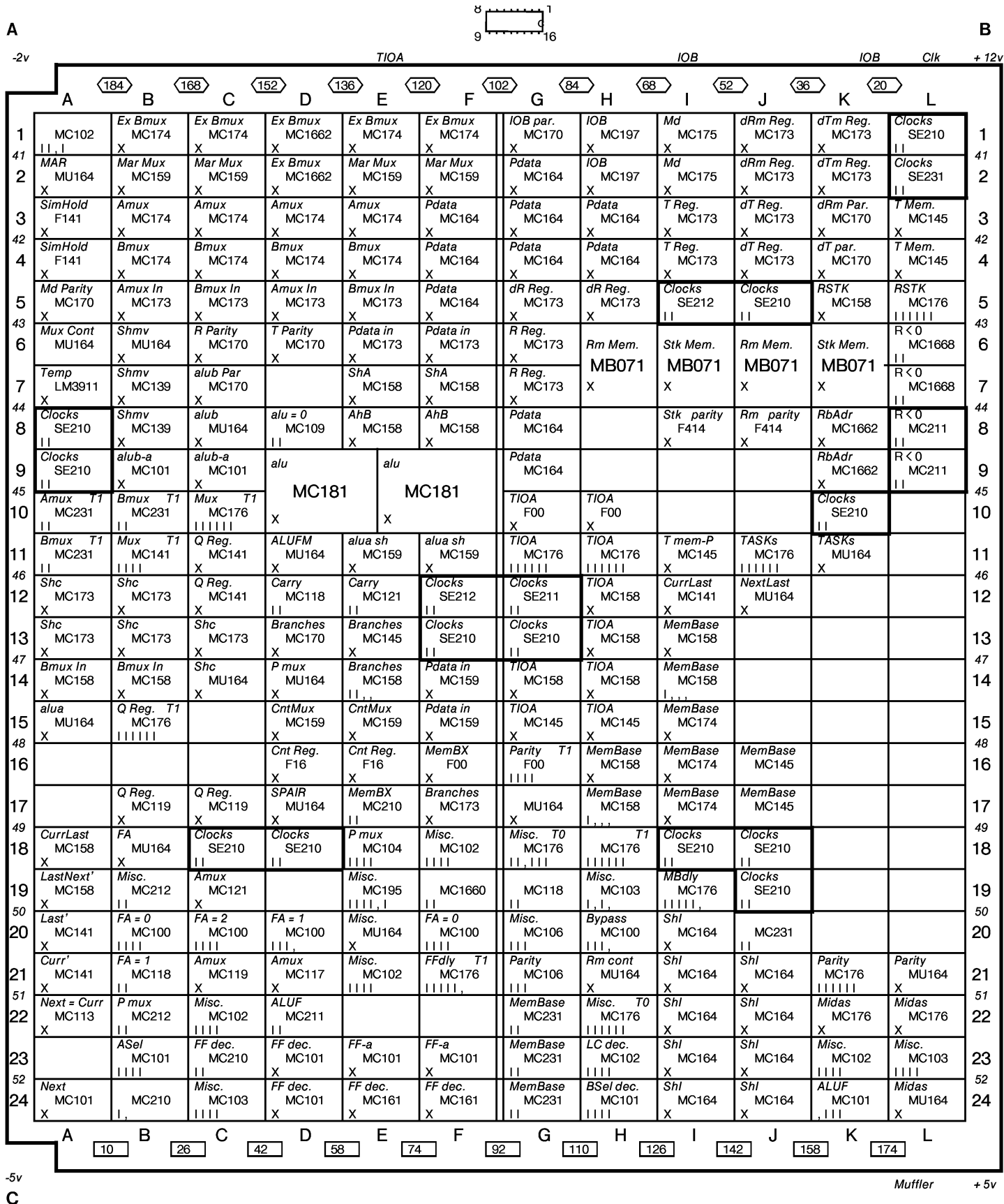
Spare Sockets for Multiwire



The following Platforms are entered so that route will cause multi-wire drill pattern to include a number of unused locations just in case they need to be used in some way.

ALUB**SHMV****SCCON****RADDR****ALUA****MAR****QPDCON****STKRB****ABCON****SPARE****ALUCON****RTSB****PERR****PRFA****NEXTCL****PJUNK**





Spare = 40

D

XEROX	Project	Reference	File	Designer	Rev	Date	Page
PARC	Dorado	Board Layout	Proch31.sil	R. Bates	Ce	6/18/79	31

Use Dorado Proms to define the following Proms:

Board Name	Prom Name	location
PorcH	Lmask (High byte)	b07
	Rmask (High byte)	b08